The Robot Science Assistant

From culture-level plans to verified robot motions, reproducible derivations and auditable executions

Technical report on the RSA kernel, its languages and its evaluation on Cell 01 of the LabAutomationTwin

[Authors and affiliations]

Generated	2026-10-17, from the pipeline's outputs
Manifest	`a8a39851a44328aac152ee46629decd9` `cddc691b1e8c2b835c422bba7cd32659`
Example	challenge–rescue experiment, design `c_d85_t48_n1`
Status	research prototype; biology is a surrogate, quantities ILLUSTRATIVE (D016)

Contents

1 Executive summary **1**

2 Background, scope and method **1**
2.1 A rational reconstruction . . . 2
2.2 Cell 01 and the laboratory twin . . . 2
2.3 The example experiment . . . 2
2.4 Method: iterate on one example . . . 3

3 Architecture **3**
3.1 Twelve kernel concepts . . . 3
3.2 Code map . . . 3

4 Knowledge, designators and constraints **3**
4.1 Action categories . . . 5
4.2 Constraints and norms . . . 6
4.3 Capabilities and measurement bindings . . . 7

5 Experimental design **7**
5.1 From design to steps in ASP . . . 7
5.2 SMT validation of the plan . . . 8

6 The lab designator language **9**
6.1 Instruments as roles . . . 9
6.2 Order, goals and threats . . . 10
6.3 Failures and handlers . . . 11
6.4 Negative controls . . . 11

7 Process models, trust, guarantees and quality **11**
7.1 Trust per step . . . 12
7.2 Repair guarantees . . . 12
7.3 Epistemic meta model . . . 13
7.4 Quality: KPIs and preference . . . 13

8 The culture-level plan **13**
8.1 Derived order . . . 16
8.2 Execution on the surrogate and what became known . . . 18
8.3 Mapping to the laboratory . . . 18

9 Execution **18**
9.1 Cross-check with the planner . . . 19
9.2 Scenarios . . . 19
9.3 Isaac Sim execution . . . 22

10 Audit trail, critic and competency questions **22**
10.1 Competency questions . . . 23
10.2 The kernel in the ontology . . . 23

11 Motion level **23**

12 Refinement proof **25**

13 Independence of the culture plan **27**

14 Reproducible derivation, plan and audit **28**

15 Findings log **29**

16 Limitations and open questions **29**

17 Roadmap **30**

A Experimentation plan tables **31**

B Competency questions **32**

C Glossary **32**

D Rebuilding this report **33**

Table 1: Main results of the RSA on the challenge–rescue example.

Result	Value	Section
Culture plan → lab plan → steps → motions	12 → 8 → 151 → 111	Section 8, Section 11
Expected / realized information gain	0.5439 / 0.357 bit (required 0.5)	Section 5, Section 8
Kernel run vs planner (nominal)	153 leaves, identical step for step; 498 audit entries	Section 9
Symbolic scenarios ending as specified	12/12	Section 9
Isaac Sim runs ending as specified	2/3 (calibration lapsed fails: F17)	Section 9.3
Competency-question checks	52/52 correct, all grounded	Section 10
LDL negative controls caught	12/12	Section 6
Handler guarantees verified	13/13	Section 7
Refinement proof	QED relative to A1–A6; O1–O7 hold	Section 12
Lab variants with identical culture plan	13/13	Section 13
Reproduction of the derivation	47/47 outputs identical; manifest `a8a39851a44328aa`	Section 14
Audit trails	15 intact, 15 tamper tests caught, 12/12 re-executions identical	Section 14

1 Executive summary

This report documents the robot science assistant (RSA): a kernel that lets a robotic laboratory carry out an experiment stated at the level of the cultures, explain what it does, prove that its robot motions realize the stated experiment, and leave a record that anyone can verify and re-execute. The assistant is developed on Cell 01, a two-arm robotic cell of a physics-based laboratory twin, with one example: a challenge–rescue organoid experiment derived for an information gain of 0.5 bit. The report describes the concepts, the languages, the knowledge, the execution system, the verification and the reproducibility machinery, and reports every result with its source file. All biological quantities come from the twin's surrogate culture model and are ILLUSTRATIVE.

What the assistant is. The RSA is a plan-based controller for a robotic laboratory, built in the tradition of CRAM and structured reactive controllers [1–3]. An experiment is written once as a plan over the cultures. The assistant derives everything below it: a design chosen by expected information gain, a partially ordered plan of laboratory action designators with contracts and failure handlers, a CRAM/RPL controller, the robot and device steps, and constraint-based motion specifications for Giskard [4]. While it executes, it keeps beliefs with provenance and freshness, checks constraints before and during every action, repairs or escalates failures, and writes a hash-chained audit entry for every lifecycle transition of every designator. After the run, a critic judges realization and validity separately, and the assistant answers competency questions about what it did and why, with the grounds of each answer.

What was shown. The table below collects the main results. Each row is computed from the outputs of the pipeline at the manifest `a8a39851a44328aa`; the section given in the last column explains how.

What was not shown. The assistant has not run in a wet laboratory. The culture is a surrogate model of the twin. Reproduction was checked on one machine. The Isaac Sim runs are recorded, not re-executed, and one of them does not end as specified; the cause (plate seating on the imager stage) is diagnosed but not repaired. The refinement proof is relative to six assumptions, two of which cover faults that the cell can detect only partly. The coraplex framework, the intended execution platform, was not yet installed; the kernel is shaped so that the port is a mapping, not a rewrite (Section 17).

How to read the report. Section 2 sets the scope and Section 3 the architecture. Section 4–Section 7 describe the knowledge, the design, the lab designator language and the process models. Section 8–Section 11 follow the experiment from the culture plan to robot motions and executions. Section 12–Section 14 cover proof, independence and reproducibility. Section 15–Section 17 list what the iterations taught, the limits and the next steps.

2 Background, scope and method

Robotic laboratories can run experiments around the clock and record everything they do [5–8]. They are usually programmed at the level of devices: a liquid-handler method, a robot program, a scheduler script. Such a program fixes *how* an experiment is executed in one laboratory but does not state *what* it does to the material, so it cannot be checked against the scientific intent, and it must be rewritten when the laboratory changes. Reproducibility suffers in both directions [9]: the intent is not recorded in a checkable form, and the record of what happened is not tied to the intent.



Figure 1: Cell 01 in the laboratory twin (Isaac Sim, path-traced): two arms, the incubator, the liquid handler in its enclosure, the bright-field imager, the tool rack and the handover nest.

The RSA addresses this with three commitments. First, the experiment is stated at the level of the cultures, independent of the laboratory. Second, every level below is derived and checked against the level above, so that the robot's motions provably realize the stated experiment, or fail safely. Third, the derivation and the execution are both recorded so that they can be verified and reproduced by someone else, in the spirit of FAIR data and data-integrity guidance [10, 11].

2.1 A rational reconstruction

The kernel is a *rational reconstruction* of earlier try-outs in the ResearchSpace material: many good ideas in mutually inconsistent forms (designator schemas, taxonomies, audit formats, verdict vocabularies). The kernel does not try to unify everything. It takes one example and keeps only what that example needs. For each concept it records where it comes from and what was changed (Table 3). It then tests whether the result *flies*: does the example run end to end, leave an audit trail, and answer competency questions correctly? Each iteration ends with findings that change the kernel or the knowledge; the findings log is in Section 15.

2.2 Cell 01 and the laboratory twin

Cell 01 is a robotic cell of a laboratory twin built on Isaac Sim with an OpenUSD scene and a semantic digital twin (SDT) in OWL. Two robots serve it: R01, a Franka FR3 that moves plates between the incubator (L10) and a handover nest, and R02, a UR5e on a rail that carries plates between the handover nest, the liquid handler (L12) in its enclosure, and the bright-field imager (L17), and couples a plate tool (T01) from an automatic tool changer. Further stations are a quarantine nest (H4), a plate hotel (L09) and a QC reference station (L21). The devices act themselves: L10 presents a plate on a shuttle, L12 doses a well, L17 loads and images. Cell 01 changes are made only through an overlay composer, so the as-built scene is never edited.

2.3 The example experiment

The example is a challenge–rescue experiment on organoid cultures. Three competing recovery models (M1 graded, M2 threshold, M3 protective) predict different effects of a rescue given after a delay. The experiment challenges two cultures, rescues one of them after the delay, keeps a third culture untreated as control, images all three, and updates the belief over the models. The design is chosen to reach an expected information gain of at least 0.5 bit at least cost (Section 5). The selected design is `c_d85_t48_n1`: challenge 0.85 au at 0 h into the rescued and the comparator culture, rescue 0.7 au at 48 h, images at 72 h. It runs in three sessions on plate P17, wells A01 (rescued), A02 (comparator) and A05 (control).

Table 2: The levels of the RSA, from the culture plan to robot motions.

Level	States	Language	Derived by	Size
Culture plan	what is done to the cultures, what becomes known	LDL (entity designators)	written	12 actions
Design	doses, delay, replicates	EIG + ASP	nested Monte Carlo, clingo	45 candidates
Lab plan	action designators with contracts and handlers	LDL	mapping, checked	8 actions
Controller	structured reactive controller	CRAM/RPL (Lisp)	generated	409 lines
Step plan	robot motions and device commands	kernel designators	clingo; kernel expansion	151 steps
Motion plan	frames, constraints, monitors, cancels	giskardpy statecharts	generated	111 motions

ILLUSTRATIVE values (D016). The culture is the twin's surrogate model; doses are in arbitrary units (au); the 600 s window outside the incubator, the 5 % dose tolerance, KPI targets and the regulation mapping are twin values. They are labelled where they appear and are not asserted as laboratory or biomedical facts.

2.4 Method: iterate on one example

- **Iteration 1** built the kernel and ran the example on a symbolic backend with fault injection: 12 scenarios, audit, critic, 27 competency questions (52 checks).
- **Iteration 2** added the Isaac Sim backend (same plan code) and expressed the kernel's categories as an OWL module checked by HermiT and Konclude.
- **Iteration 3a** introduced the lab designator language (LDL) in three versions (v0.1–v0.3), each after a round of design decisions, then the culture-level plan, its grounding in Cell 01, the Giskard motion level, the independence check, the refinement proof, and the reproducibility machinery.
- **Iteration 3 (pending)** ports the kernel to coraplex.

3 Architecture

The assistant is organised in levels (Table 2). Each level states something different, in its own language, and is derived from the level above. The culture plan is written; everything else is generated and checked.

Inside the kernel, seven components cooperate (Fig. 2). The knowledge base answers resolution queries with their grounds. The plan library builds sub-plans of each action category at perform time. The executive drives designators through their lifecycle, projects plans before anything moves, applies the understanding gate and the policies, and handles failures. A backend executes leaves: a symbolic world with fault injection, or the Isaac Sim twin. Every transition goes into the audit trail, from which the critic and the competency-question engine work.

3.1 Twelve kernel concepts

Table 3 lists the twelve concepts the example needs, each with its kernel definition and origin. Three of them are new relative to the sources. The *device-command designator* is a fourth leaf kind beside motions, because Cell 01's devices are co-actors. The *event designator* is the audit-side record of what happened. And *freshness* is part of belief, not only provenance (finding F2).

3.2 Code map

The kernel is a Python package (`rsa/`) of about 7,580 lines. Its API is shaped like coraplex: designators are dataclasses, action designators build their plans, and plans are trees of sequential, try-in-order and monitor nodes with action, motion and command leaves.

4 Knowledge, designators and constraints

One source of knowledge. The planner, the resolver, the monitors and the competency questions read the same facts. They are generated from the OKF ontology ABoxes of the lab (SDT, safety, experiments) into ASP facts (`build/e2m/facts_<tag>.lp`), complemented by the Cell 01 component records (capabilities, contracts, monitors, failure modes; read-only) and the kernel registries (categories, constraints, norms, devices, scenarios, competency

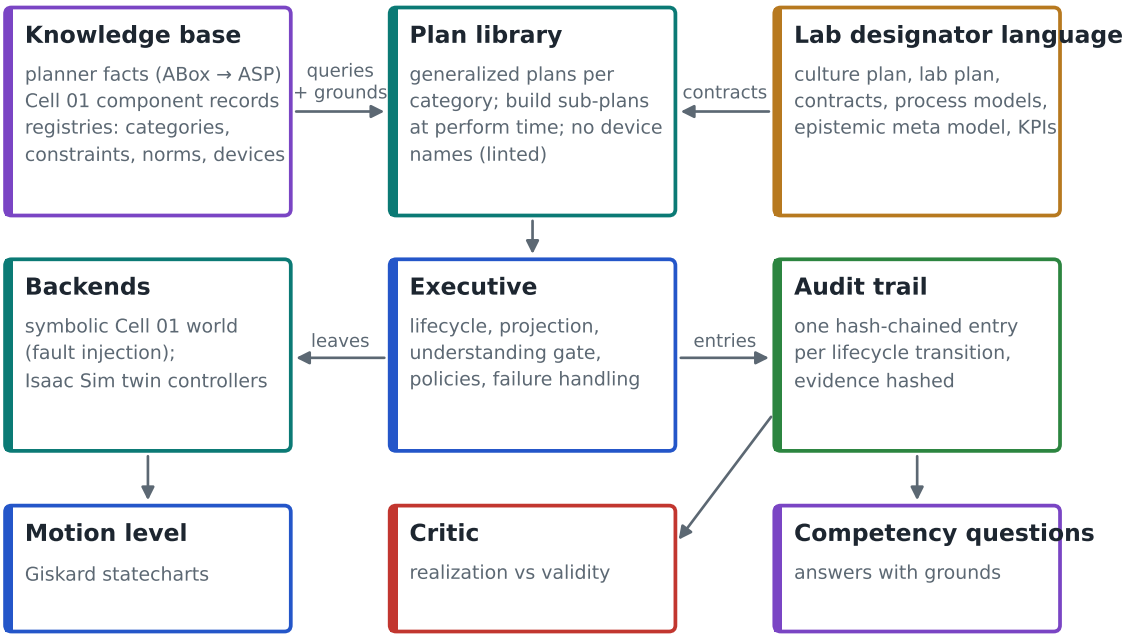


Figure 2: Kernel components and their data flow. The plan code never names a device; devices enter only through resolution against the knowledge base.

Table 3: The twelve kernel concepts, their definitions and where they come from.

#	Concept	Kernel definition	Taken from
1	Designator	kind, category, determiner, bindings, constraints; kinds action, object, location, motion, device-command, event	HADL; KnowRob; coraplex
2	Action category	a card owned by the category: level, parent, effect class, facet, roles, capability, pre/effects, sub-plan, monitors, recovery, evidence	KnowRob; Cell 01 schemas
3	Category hierarchy	scientific ⟶ lab task ⟶ manipulation ⟶ motion, device-command beside motion; typed by effect class and facet	AICOR effect partition; Cell 01
4	Generalized plan	code of a category that builds the sub-plan from a bound designator at perform time, querying the knowledge base	CRAM; coraplex
5	Resolution	a query returns value, grounds (fact atoms used) and rejected alternatives with reasons	KnowRob resolution; certificates
6	Knowledge base	the planner's facts, the Cell 01 component records and the kernel registries; one source for all consumers	e2m derivation; Cell 01 SDT
7	Capability	a verb qualified by state and by measurement binding (no match ⊂ capability match ⊂ provisional ⊂ validated)	Cell 01; RSV binding lattice
8	Constraint	registry entry: id, column (realization or validity), source, text, applies-to, check, ASP rule, z3 theorem	uc_ccr; e2m S1–S9
9	Lifecycle	open ⟶ grounded ⟶ accepted ⟶ executing ⟶ executed | failed ⟶ repaired | abstained | escalated	Cell 01 schemas
10	Commitment record	one hash-chained entry per lifecycle transition, with bindings, grounds, verdicts, monitors and evidence	uc_ccr audit; NEEM schema
11	Two-column verdict	realization (ran correctly and safely) ⊥ validity (result interpretable); commit only if both hold	uc_ccr critic
12	Understanding gate + CQs	an incomplete understanding frame is not accepted; competency questions answered from the runtime trail	Cell 01 understanding contract

Table 4: Modules of the RSA kernel and pipeline.

Module	Lines	Role
`rsa/kb.py`	260	knowledge base: planner facts, component records, registries; queries return grounds
`rsa/core.py`	420	designators, plan nodes, belief with provenance and freshness, executor
`rsa/library.py`	405	generalized plans of the categories; no device names (linted)
`rsa/checks.py`	213	constraint checks, policies (window, airspace), understanding frame
`rsa/backend.py`	390	symbolic Cell 01 world with fault injection
`rsa/isaac.py`	558	Isaac Sim backend: leaves bound to the twin's controllers
`rsa/audit.py`	86	hash-chained commitment records and verifier
`rsa/critic.py`	92	two-column verdict from the trail alone
`rsa/cq.py`	436	competency questions answered from trail, knowledge and dry runs
`rsa/ldl.py`	839	LDL parser, checker: order, threats, instruments, handlers, trust, KPIs
`rsa/ldl_trust.py`	452	process models, guarantees, epistemic meta model
`rsa/ldl_to_cram.py`	319	CRAM/RPL controller generation
`rsa/culture_plan.py`	361	culture plan: order, surrogate execution, mapping, feasibility
`rsa/grounding.py`	260	instantiation, trips, waits, computations, reactions
`rsa/motion_frames.py`	210	goal frames from the scene; inverse kinematics reachability
`rsa/giskard_motions.py`	473	Giskard motion statecharts
`rsa/refinement_proof.py`	269	refinement proof O1–O7 and negative controls
`rsa/independence.py`	230	culture plan in 13 lab variants
`rsa/reproduce.py`	204	content-addressed manifest; verify by re-run
`rsa/protocol.py`	181	experimentation plan and audit certificate

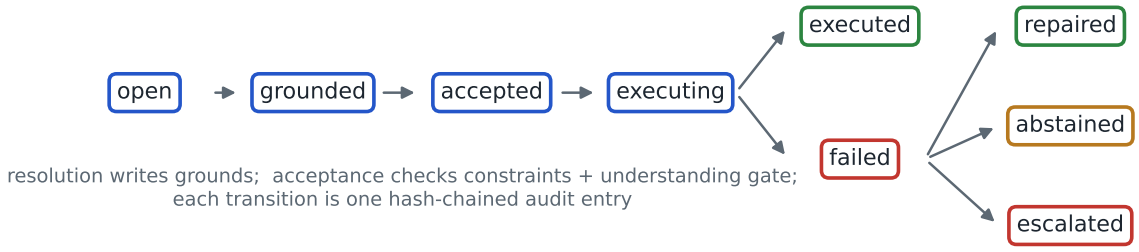


Figure 3: Designator lifecycle. Every transition is one audit entry; repairs are recorded on the node that repaired.

questions). Two derivations from the same facts, the clingo plan and the kernel's expansion, must agree; they do, step for step (Section 9).

Resolution with grounds. A resolution query returns a value, the fact atoms it used, and the alternatives it rejected with the reason. For the dosing step, the instrument resolves to L12's plate nest with grounds `has_function(l12,dispensing)`, `dose_well` among L12's capabilities and `seat(l12, …)`; five other devices are rejected, each with the missing function or capability. These records go into the audit trail, so the assistant can later answer why it chose a device.

Designator lifecycle. Each designator moves through the lifecycle of Fig. 3. Plan-node status (created, running, succeeded, failed), designator lifecycle and episode verdict are three separate vocabularies (decision K5); conflating them was a source of inconsistency in the earlier material.

4.1 Action categories

The category registry has 43 cards on five levels (Table 5). A card belongs to the category, not to the instance: level, specialisation parent, refinement parts, effect class (the AICOR partition: object displacement, substance manipulation, perception, connecting, configuration change, regulation, maintenance), facet (transport, operation, transformation), roles, required capability, pre- and post-conditions as constraint ids, sub-plan, monitors, recovery and evidence obligations. The instance only binds roles. Expressed as an OWL module, the cards first produced six unsatisfiable classes, because `parent` mixed specialisation, partonomy and realisation (F14); separating `parent` from `parts` made the module consistent.

Table 5: Action categories by level (kernel registry).

Level	#	Categories
scientific	4	`experiment`, `wait`, `intervene`, `measure`
lab task	8	`run_session`, `retrieve`, `store`, `transport_lab`, `dose`, `image`, `calibrate`, `quarantine`
manipulation	6	`transporting`, `picking_up`, `placing`, `coupling_tool`, `presenting`, `stowing`
device command	16	`check_interlocks`, `verify_identity`, `present`, `stow`, `wait_environment`, `store_cmd`, `tip_pick`, `aspirate`, `dispense`, `tip_eject`, `load`, `image_well`, `autofocus`, `incubate`, `contain`, `validate`
motion	9	`approach`, `insert`, `grasp`, `lift`, `transit`, `place`, `release`, `retract`, `tool_couple`

4.2 Constraints and norms

Constraints are registry entries with a column. 14 runtime constraints guard the transitions of the categories they apply to; 7 validity checks (V1–V7) are evaluated by the critic over the whole episode (Table 6). Every constraint names its source: an ontology axiom, a Cell 01 monitor, an e2m axiom or a kernel finding.

Table 6: Constraint registry: runtime constraints and the critic's validity checks.

Constraint	Column	Applies to	Statement	Source
`interlocks_closed`	realization	run_session	enclosure interlocks are closed before a trip starts	safety.ofn enclosure_door_interlock; Cell 01 safety enclosure
`projection_succeeds`	realization	experiment, run_session	the plan can be projected to completion (every designator groundable) before anything is moved – for the whole experiment before the first intervention, and again for each session before the plate leaves its home	CRAM projection / Ground & Emulate gate (cram_viz); kernel iteration …
`window_prospective`	realization	run_session	projected trip duration plus margin fits the window outside the incubator	e2m E2M8 S5 (600 s, 20 % margin; ILLUSTRATIVE window)
`window_kept`	realization	run_session	elapsed time outside the incubator stays within the window	Cell 01 monitor time_outside_environment
`identity_verified`	realization	retrieve	the plate identity read at the storage device matches the plate the session is about	safety.ofn identity_verified; Cell 01 monitor plate_identity
`environment_before_store`	realization	store	the incubator environment is accepted before the plate is stored	safety.ofn l10_environment_valid
`l12_airspace_clear`	realization	dose	no robot is in L12's deck airspace while L12 moves	safety.ofn l12_deck_airspace
`calibration_before_imaging`	realization	image	the imager is believed calibrated, or a QC calibration precedes the imaging	safety.ofn l17_calibration_valid
`tool_compatible`	realization	coupling_tool	a tool that contacts culture is compatible with the culture before coupling	Cell 01 contamination-and-asepsis
`contamination_free`	realization	experiment, dose, transport_lab	no challenge or rescue is performed on a plate under suspected contamination	safety rule contamination_suspected_blocks
`reagent_bound`	realization	dose	the reagent is bound to a rack position with identity and lot before aspiration	Cell 01 L12 constraint 'Correct reagent identity/lot must be bound be…
`dispense_completed`	realization	dose	dispense completed with the commanded volume within tolerance	Cell 01 monitor dispense_completion; tolerance 5 % ILLUSTRATIVE
`challenge_requires_controls`	validity	experiment	a challenge is authorised only if the design contains a control arm	safety rule challenge_requires_controls; uc_ccr negative control (NR3)
`measurement_binding_validated`	validity	image	the imager's binding for the readout is validated (not only a capability match)	RSV measurement-binding lattice; VRB CF2
`V1_identity_continuity`	validity	episode	every identity read in the run is the experiment's plate	Cell 01 critic identity; D009
`V2_calibration_at_measurement`	validity	episode	the imager was calibrated (observed) when each readout was acquired	Cell 01 monitor calibration_state
`V3_binding_validated`	validity	episode	every readout comes from a validated measurement binding	RSV binding lattice
`V4_controls_present`	validity	episode	the control well was measured	uc_ccr NR3
`V5_dose_delivered`	validity	episode	each designed dose was delivered within tolerance (volume; not biological effective dose)	Cell 01 L12 'Executed volume is not equivalent to biological effectiv…
`V6_sample_integrity`	validity	episode	every trip kept the window outside the incubator	Cell 01 monitor time_outside_environment

Table 7: Regulation clauses and what realises them (ILLUSTRATIVE mapping).

Clause (ILLUSTRATIVE mapping)	Topic	Realised by
EU-GMP-Annex11-9	audit trail of GMP-relevant changes and actions	audit_field: hash_chain, lifecycle_entry_per_transition
ALCOA-attributable	data attributable to who/what produced it	audit_field: agent, software
ALCOA-contemporaneous	recorded when it happened	audit_field: t_sim
ALCOA-original	original record / true copy preserved	audit_field: evidence_sha256
21CFR11-11.50	signature manifestations (who, when, meaning)	documentary: quarantine_release_signoff
OECD-GLP-study-plan	study plan approved before the study starts	documentary: study_plan_approval
ISO10218-2-safeguarding	safeguarding of the robot cell	runtime: interlocks_closed
biosafety-contamination	no handling of potentially contaminated cultures without containment	runtime: contamination_free

Constraint	Column	Applies to	Statement	Source
`V7_design_realised`	validity	episode	every intervention and measurement of the design was executed at its designed culture time	e2m workflow (sessions)

Regulation clauses are cited by identifier only and mapped to what realises them in the kernel: an audit field, a runtime constraint, or a documentary obligation the kernel cannot discharge (Table 7). The mapping is ILLUSTRATIVE. Its point is that the assistant can say which clauses a run covers and which it does not (competency question CQ-19).

4.3 Capabilities and measurement bindings

A capability is a verb from a component record, qualified by state (calibrated, tool coupled, tips present, automated) and by a measurement binding on a lattice: no match ⊂ capability match ⊂ provisional ⊂ validated. The distinction matters for validity. An imager that *can* image a well but whose binding for the readout is not validated produces an image that does not count as evidence. This is how the `imager_swapped` scenario ends realised but not valid (NR3), while `imager_incapable` is refused before anything moves.

5 Experimental design

The design answers: which dose, delay and number of replicates separate the three recovery models best, at least cost? The assistant uses Bayesian experimental design [12, 13]: the expected information gain (EIG) of a design is the expected reduction of entropy of the belief over the models, estimated by nested Monte Carlo over the twin's calibrated surrogate. The planner then selects, among candidates meeting the required gain, the least-cost one that can be executed safely in Cell 01.

5.1 From design to steps in ASP

The derivation from the selected design to the grounded steps is an answer-set program [14] that executes the experiment-to-motion axioms E2M1–E2M8. The stable model *is* the plan: the selected experiment, the wells, the

Table 8: The design problem and its solution.

Item	Value
question	which recovery model (M1 graded, M2 threshold, M3 protective) explains the rescue efficacy after a delay
candidates evaluated	45 (dose × delay × replicates); 13 meet the 0.5-bit requirement
EIG estimator	nested Monte Carlo, 800 samples, seed 7, uniform prior over 3 models (max 1.585 bit)
selected design	`c_d85_t48_n1`: challenge 0.85 au, rescue 0.7 au after 48 h, 1 replicate
expected information gain	0.5439 bit ≥ 0.5 bit
wells	rescued A01, comparator A02, control A05
cost (least, then max EIG)	9986 decisecond-equivalents (robot / device time + 60 s per well)
clingo	332 atoms, 60750 rules, optimal

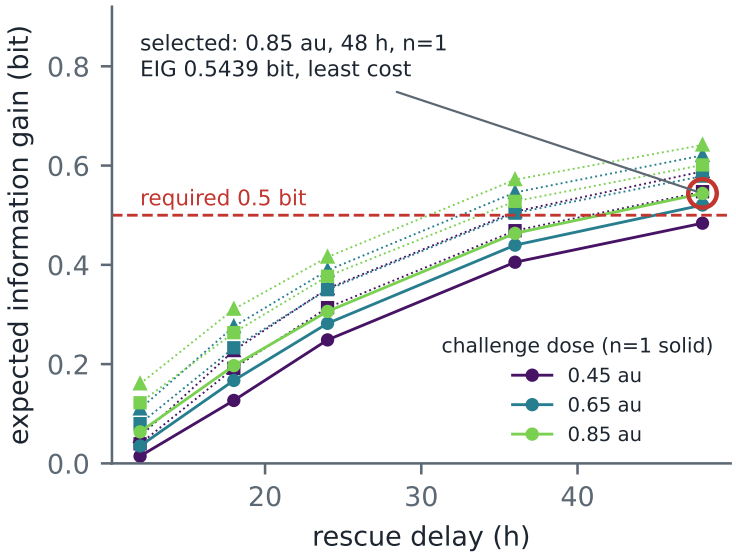


Figure 4: Expected information gain of the candidate designs by delay and dose. The selected design is circled.

```
% ------------------------------------------------------------------------------------------------------ E2M1
↪  selection
{ selected(C) : cand(C,_,_,_,G), req(Q), G >= Q } = 1.
sel(D,Dt,N,G) :- selected(C), cand(C,D,Dt,N,G).

% ------------------------------------------------------------------------------------------------------ E2M3
↪  arms/units
{ role(W,rescued)    : free(W,_) } = N :- sel(_,_,N,_).
{ role(W,comparator) : free(W,_) } = N :- sel(_,_,N,_).
{ role(W,control)    : free(W,_) } = 1.
:- role(W,R1), role(W,R2), R1 != R2.
% symmetry breaking (wells are interchangeable units): the lowest free indices, rescued < comparator < control
used(W) :- role(W,_).
:- used(W2), free(W2,X2), free(W1,X1), X1 < X2, not used(W1).
rorder(rescued,1). rorder(comparator,2). rorder(control,3).
:- role(W1,R1), role(W2,R2), free(W1,X1), free(W2,X2), X1 < X2, rorder(R1,O1), rorder(R2,O2), O1 > O2.
treated(W) :- role(W,rescued).
treated(W) :- role(W,comparator).

% ------------------------------------------------------------------------------------------------------ E2M4
↪  sessions
session(s0, 0).
session(s1, Dt) :- sel(_,Dt,_,_).
session(s2, 72).
mat(s0, dose(W,challenge,D)) :- sel(D,_,_,_), treated(W).
mat(s1, dose(W,rescue,R))    :- rescue_au(R), role(W,rescued).
mat(s2, image(W))            :- role(W,_).
kind(dose(W,X,Y), dose) :- mat(_, dose(W,X,Y)).
kind(image(W), image)   :- mat(_, image(W)).
well_of(dose(W,X,Y), W) :- mat(_, dose(W,X,Y)).
well_of(image(W), W)    :- mat(_, image(W)).
```

Listing 1: Excerpt of the experiment-to-motion axioms in ASP (selection, arms, sessions).

sessions, the trips, the routes and the ordered robot and device steps of every trip. Facts such as seats, shuttles, tools and durations come from the ontology; durations are apportioned from the twin's run traces. The excerpt in Listing 1 shows design selection, arm assignment with symmetry breaking, and the material of each session.

5.2 SMT validation of the plan

An independent validator encodes the plan in z3 [15] and proves the theorems V1–V5: executability and preservation of preconditions over all steps, safety rules, the time window with a ±20 % duration tolerance, realization of the

Table 9: The three sessions of the derived plan, with planned and Isaac Sim durations.

Session	Culture time	Device	Material	Steps	Planned (s)	Isaac (s)
s0	0 h	L12	dose A01 challenge 0.85 au; dose A02 challenge 0.85 au	52	298.9	305.7
s1	48 h	L12	dose A01 rescue 0.70 au	48	263.3	230.1
s2	72 h	L17	image A01; image A02; image A05	51	256.4	207.3

Table 10: SMT theorems over the derived plan (`validate_plan_z3.py`).

Theorem	Holds	Detail
V1 executability / preservation	yes	checked=278
V2 safety S1 S2 S3 S6 S7	yes	checked=115
V3 time window (S5), durations +-20 %	yes	checked=3
V4 realisation (doses, images, session times)	yes	–
V4b arms E2M3 + control S9	yes	–
V5 gain met (EIG >= g)	yes	eig_bits=0.5439, required_bits=0.5
V5b least cost, then max EIG, over all candidates meeting g	yes	smt=unsat, compared=13

design (doses, images, session times), the arms with a control, the information gain, and least cost over all candidates meeting the requirement. 10 negative controls (mutated plans) are refuted. Finding F11 came from this layer: a QC calibration placed after the plate occupied the imager stage was physically infeasible in both derivations; the validator now refutes the old plan at step 24.

6 The lab designator language

The lab designator language (LDL) states *what* a laboratory action must bring about, never *how*. Every term is an entity description (`determiner sort slot*`) over lab entities only: plates, wells, substances, designs, arms, environments, excursions, transfers, measurements, records, instruments, persons and requests. A condition is an entity description too: once its variables are bound it is a ground query with a truth value. No robot, gripper, tool, pose, frame, trajectory or device identity can be written; a lint makes the boundary checkable.

Semantics. An execution trace satisfies an action iff its `requires` hold at the start, nothing its `prevents` describes occurs during the action, its `maintains` hold throughout, and its `achieves` hold at the end. `prevents` and `maintains` of an enclosing action are inherited by every sub-action; `achieves` is not. A condition φ marked `known` holds iff φ is believed with observed, fresh provenance; a plain `requires` may be established by the executive (insert a calibration), a `known` one may only be verified (identity cannot be established, only read). **Admissibility** sharpens the understanding gate: an action is accepted only if every term resolves, every `prevents` item has a monitor that can detect it, every `maintains` item a monitor or regulator, every `achieves` item an evidence path, and every `requires` item can be verified or established. A `prevents` clause that nothing can observe is not a constraint, so the action is refused.

6.1 Instruments as roles

Instruments appear only attributively, but they fill a role: a slot the plate is inserted into and a process that, once started, performs the action. Realization is machine tending: insert, start, await, remove. The checker resolves every

Table 11: LDL clauses and their operational obligations.

Clause	Meaning over the action's interval	Operational obligation
`requires`	holds (and is known to hold) at the start	acceptance check, or a reason to establish it first
`prevents`	no matching state or event occurs during the action	a detector and a reaction (abort, contain)
`maintains`	holds throughout the action	a regulator with a recorded history
`achieves`	holds at the end and persists until changed	post-condition with an evidence record
`goal`	the effect the action is for; links steps causally	justified by the experiment's goal
`signals`	the failures the action may raise	every failure needs a handler in the control program
`scope`	a non-terminating prevents/maintains opened and closed by fluents	a monitor around the realization

```
(contract intervening
  (roles (into (the well ?w (of ?p))) (substance ?s) (dose ?q) (at (culture-time ?t)))
  (instrument (an instrument ?d
    (slot (a plate-slot (accepts ?p)))
    (process (a process (type substance-transfer) (into ?w) (of ?s) (amount ?q)))))
  (goal (the well ?w (received ?s ?q)) :tolerance 5 % :column validity)
  (requires
    (known (the plate ?p (identity verified))) :consequential
    (known (some substance ?s (identity verified))) :consequential
    (believed (the plate ?p (contamination none)) (grade (source inferred))) :consequential
    (the plate ?p (culture-time ?t)))
  (prevents
    (an event (type contamination-introduced) (of ?p)))
  (scope (begins-when (a portion ?u (of ?s) (drawn-for ?self)))
         (ends-when (the portion ?u (delivered true)))
    (prevents (a transfer (of ?u) (into (a well (of ?p) (not ?w))))))
  (achieves
    (a transfer (of ?s) (into ?w))
    (a record (of ?self) (states ?w ?s ?q ?t) (original true)))
  (signals
    (a failure (type identity-mismatch))
    (a failure (type dose-not-delivered))
    (a failure (type contamination-suspected))))
```

Listing 2: The contract of the action type `intervening` (`registry/lab/contracts.ldl`).

Table 12: Instrument roles of the lab plan and how they resolve in Cell 01.

Step	Process	Function / capability	Device and slot	Rejected
?x1	substance-transfer	dispensing / `dose_well`	L12 `stations_l12_liquidhandler_plate_nest`	5
?x2	substance-transfer	dispensing / `dose_well`	L12 `stations_l12_liquidhandler_plate_nest`	5
?w1	incubation	incubation / `incubate_plate`	L10 `stations_l10_incubator_transfer_station`	5
?x3	substance-transfer	dispensing / `dose_well`	L12 `stations_l12_liquidhandler_plate_nest`	5
?w2	incubation	incubation / `incubate_plate`	L10 `stations_l10_incubator_transfer_station`	5
?m1	imaging	imaging / `image_well`	L17 `stations_l17_brightfieldimager_plate_load_frame`	5
?m2	imaging	imaging / `image_well`	L17 `stations_l17_brightfieldimager_plate_load_frame`	5
?m3	imaging	imaging / `image_well`	L17 `stations_l17_brightfieldimager_plate_load_frame`	5

role against the knowledge (Table 12): an automated device with a plate seat, with the function and capability verb of the process.

6.2 Order, goals and threats

The order of the lab plan is derived, not written. From goals and `achieves` the checker builds 13 causal links; 9 threats on culture time are all resolved by promotion, leaving 7 Hasse edges and 12 linear extensions, in the manner of least-commitment planning [16]. Every step goal is justified by the experiment's goal through the links. A `scope` bounds a non-terminating prevents-item by fluents: the cross-dosing prevents-item holds only while the portion drawn for the action is in the tip, which removed a spurious mutual exclusion between dosing A01 and A02 that v0.1 produced.

```
(an action ?x3
    (type intervening)
    (into ?r)
    (substance ?rs)
    (dose 0.70 au)
    (at (culture-time 48 h)))
```

Listing 3: A lab action designator of the generated plan (the rescue dose).

```
(handles                                          ; every handler states the guarantee it claims; rsa/ldl.py
↪  checks it
  (on (a failure (type identity-mismatch))
      (do (an action (type quarantining) (material (the plate (identity mismatched)))))
      (then escalate)
      (guarantees safe-state contained))
  (on (a failure (type measurement-invalid) (cause calibration_expired))
      (do (re-establish (an instrument (function imaging) (calibration current))))
      (then replan) (else escalate)
      (guarantees goal-or-safe-state))
  (on (a failure (type measurement-invalid) (cause out_of_focus))
      (do (re-establish (the plate ?p (seated true))))
      (then (retry (max 1))) (else escalate)
      (guarantees goal-or-safe-state))
  (on (a failure (type measurement-invalid) (cause plate_misseated))
      (do (re-establish (the plate ?p (seated true))))
      (then (retry (max 1))) (else escalate)
      (guarantees goal-or-safe-state))
  (on (a failure (type validity-effect-unachievable))
      (then escalate)
      (guarantees safe-state))
  (on (a failure (type dose-not-delivered) (cause wrong_reagent))
      (do (re-establish (some substance ?s (identity verified))))
```

Listing 4: Handlers of the experiment's control program (excerpt of `challenge_rescue.ldl`).

Table 13: Negative controls of the LDL checker; each mutation must be caught.

Negative control (mutation)	Caught by
N1 design anchors removed	?w1 threatens start -> ?x1 ((the plate ?p (culture-time 0 h))); ?w2 threatens start -> ?x1 ((the plate ?p (culture-time…
N2 control well dosed	VIOLATED by ['?x9']
N3 design without a control arm	?k; VIOLATED
N4 device named in lab text	device or robot id 'l17' in (on l17)
N5 rescue at a culture time nothing achieves	?x3: (the plate ?p (culture-time 36 h))
N6 identity-mismatch handler removed	?m1: identity-mismatch unhandled; ?m2: identity-mismatch unhandled
N7 instrument process no device offers	?m1: ['h4', 'no device function realises process sequencing']; ?m2: ['h4', 'no device function realises process sequenc…
N8 a step whose goal serves nothing	?m9 unjustified
N9 focus retry without re-seating (F17)	out_of_focus is persistent and the handler does not re-establish (the plate ?p (seated true)): a retry meets the same c…
N10 process fault not in the component record	L12_LiquidHandler: ["fault pipette_crash not in the record's failure_modes"] unmodelled ['well_misregistration']
N11 repair claims the goal after the irreversible phase	well_misregistration occurs at dispense, at or after the irreversible phase dispense; the goal cannot be re-achieved
N12 meta model without rules for weak consequential beliefs	(the plate ?p (identity verified)): ['inferred', 'assumed'] untreated

6.3 Failures and handlers

Each contract lists the failures it may signal. The experiment's control program handles them: `(on (a failure …) (do …) (then retry | replan | escalate) (else …) (guarantees …))`. All 21 signalled failures of the plan have a handler. Listing 4 shows the handlers; Table 43 in the appendix lists them all.

6.4 Negative controls

A checker that accepts everything proves nothing. Each of the 12 mutations in Table 13 breaks one property, and the checker must refuse it with the right reason.

7 Process models, trust, guarantees and quality

The trustworthiness of an experiment depends on what happens *inside* the devices. LDL v0.3 therefore models each laboratory process: its phases, the phase after which it cannot be undone, and its faults. Each fault states the failure it signals, the goal it corrupts, the monitor that detects it and with what coverage, whether a repeat meets the

```
(process-model substance-transfer
  (realised-by (an instrument (function dispensing)))
  (component L12_LiquidHandler)
  (contract dose_well)
  (phases verify-reagent pick-tip aspirate dispense eject-tip)
  (irreversible-at dispense)
  (produces (the well ?w (received ?s ?q)))
  (faults
    (a fault (name wrong_reagent) (signals dose-not-delivered) (at verify-reagent)
      (corrupts (the well ?w (received ?s ?q)))
      (detected-by (a monitor (control "reagent identity check"))) (coverage full)
      (persistence persistent) (removed-by (some substance ?s (identity verified)))
      (record-recovery "abort before aspiration; quarantine if dispense occurred"))
    (a fault (name tip_missing_or_dropped) (signals dose-not-delivered) (at pick-tip)
      (corrupts (the well ?w (received ?s ?q)))
      (detected-by (a monitor (observation "tip-presence sensor"))) (coverage full)
      (persistence transient)
      (record-recovery "abort; replace tip; evaluate plate exposure"))
    (a fault (name clog_or_air_aspiration) (signals dose-not-delivered) (at aspirate)
      (corrupts (the well ?w (received ?s ?q)))
      (detected-by (a monitor (observation "pressure trace"))) (coverage full)
      (persistence transient)
      (record-recovery "mark dose uncertain; repeat only if protocol allows"))
    (a fault (name well_misregistration) (signals dose-not-delivered) (at dispense)
      (corrupts (the well ?w (received ?s ?q)))
      (detected-by (a monitor (observation "nest sensor + vision"))) (coverage partial)   ; sees a misseated
      ↪  plate, not the dispense position
      (persistence persistent) (removed-by (the plate ?p (seated true)))
      (record-recovery "quarantine affected plate; do not silently correct provenance")))
  (evidence "reagent lot" "tip ID/type" "volume" "pressure trace" "well identity")
  (must-not-assume "commanded volume equals delivered volume" "delivered dose equals biological injury"))
```

Listing 5: Process model of substance transfer (L12), `registry/lab/processes.ldl`.

Table 14: Process models and their grounding in the Cell 01 components.

Process	Component	Phases	Irreversible	Faults	Must not assume
substance-transfer	L12_LiquidHandler	verify-reagent → pick-tip → aspirate → dispense → eject-tip	dispense	wrong_reagent, tip_missing_or_dropped, clog_or_air_aspiration, well_misregistration	commanded volume equals delivered volume; delivered dose equals biological injury
imaging	L17_BrightFieldImager	load → autofocus → acquire → assess → export	none	calibration_expired, plate_misseated, out_of_focus	morphology image directly equals biological state
incubation	L10_Incubator	store → hold → release	hold	temperature_excursion, CO2_excursion, gate_stuck_open	nominal setpoint equals actual culture exposure without sensor evidence
containment	H4_Quarantine	receive → isolate → hold	none	unauthorized_release	quarantine implies biological failure

same cause (persistence), and which re-established condition removes it. The models are grounded in the Cell 01 component records: every fault must be one of the record's failure modes, every monitor one of its observations.

7.1 Trust per step

Reasoning over the models gives the trust profile of each step (Fig. 5, Table 15). 21 of 27 fault occurrences are fully detected. Two fault kinds are only partly detected: well misregistration at L12, which occurs at the irreversible dispense, and plate misseating at L17, the lateral shift behind finding F17. They are the two trust gaps of the current cell and appear as assumption A2 of the refinement proof.

7.2 Repair guarantees

Every handler states what it guarantees, and the checker verifies the claim. *goal* needs the fault to be detected before the irreversible phase and either transient or removed by what the handler re-establishes. *safe-state* needs the final branch to escalate or abstain, the plate to be restorable, nothing committed and the evidence kept. *contained* needs a

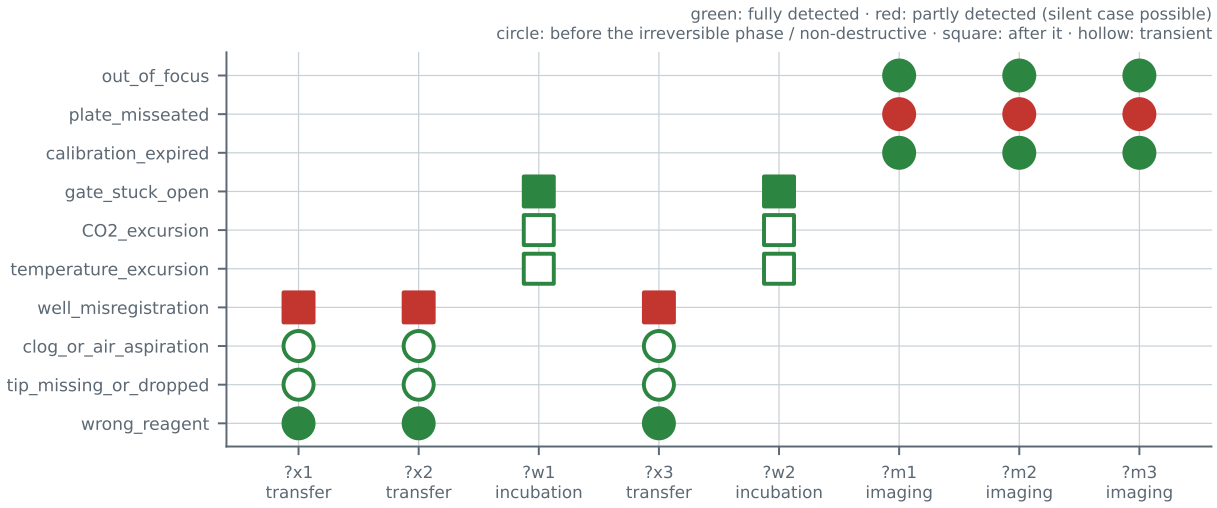


Figure 5: Trust profile of the lab plan: fault × step, by detection coverage, timing relative to the irreversible phase, and persistence.

Table 15: Faults of the process models: detection, timing, persistence and handling.

Fault	Process	Detected by	Coverage	Timing	Persistence	Handled
`wrong_reagent`	substance-transfer	reagent identity check	full	before the irreversible phase	persistent	yes
`tip_missing_or_dropped`	substance-transfer	tip-presence sensor	full	before the irreversible phase	transient	yes
`clog_or_air_aspiration`	substance-transfer	pressure trace	full	before the irreversible phase	transient	yes
`well_misregistration`	substance-transfer	nest sensor + vision	partial	after the irreversible phase	persistent	yes
`temperature_excursion`	incubation	temperature sensor	full	after the irreversible phase	transient	yes
`CO2_excursion`	incubation	CO2 sensor	full	after the irreversible phase	transient	yes
`gate_stuck_open`	incubation	gate position sensor	full	after the irreversible phase	persistent	yes
`calibration_expired`	imaging	calibration record	full	non-destructive process	persistent	yes
`plate_misseated`	imaging	nest sensor + pose	partial	non-destructive process	persistent	yes
`out_of_focus`	imaging	computed focus score	full	non-destructive process	persistent	yes

quarantining action with a containment goal. The checker rejects a focus retry that does not re-seat the plate (N9) and a goal claimed after the irreversible dispense (N11).

Against the recorded runs, every repair event kept its claim (Table 17). In the Isaac focus case the kernel did not repair as prescribed: it refocused instead of re-seating.

7.3 Epistemic meta model

Beliefs carry a grade: source (observed > reported > inferred > assumed > predicted > unknown) × freshness × corroboration. A meta model, itself written in LDL, says how each shortfall is treated depending on the item, its use (consequential or routine) and the action context. A treatment is a plan with a cost: re-observe, verify before commit, contain and escalate, sense or abstain, plan only. The 10 rules give every requirement a treatment for every shortfall.

7.4 Quality: KPIs and preference

Hard limits stay in `prevents`; soft targets are KPIs, each an aggregate over an entity description, ranked lexicographically. Safety, reliability and auditability lead, then excursion time, cost, dose deviation and time to result. Targets and cost weights are ILLUSTRATIVE.

8 The culture-level plan

The culture plan states the experiment over the cultures only. It names the question, the entities (three cultures by role, two substances, a belief over the three models), the goal, what is given, what must never happen (the control is

Table 16: Handler guarantees and their verification.

Failure	Cause	Claims	Steps	Holds
identity-mismatch	any	safe-state, contained	6	yes
measurement-invalid	calibration_expired	goal-or-safe-state	3	yes
measurement-invalid	out_of_focus	goal-or-safe-state	3	yes
measurement-invalid	plate_misseated	goal-or-safe-state	3	yes
validity-effect-unachievable	any	safe-state	3	yes
dose-not-delivered	wrong_reagent	goal-or-safe-state	3	yes
dose-not-delivered	tip_missing_or_dropped	goal-or-safe-state	3	yes
dose-not-delivered	clog_or_air_aspiration	safe-state	3	yes
dose-not-delivered	well_misregistration	safe-state, contained	3	yes
contamination-suspected	any	safe-state, contained	3	yes
environment-excursion	any	safe-state	2	yes
containment-breach	any	safe-state	0	yes
prevents-violated	any	safe-state	0	yes

Table 17: Repairs in the recorded runs against the handlers' claims.

Run	Failure	Handler	Repair	As prescribed	Claim held
calibration lapsed	calibration_invalid	measurement-invalid/calibration_expired	transport_lab, calibrate	yes	yes
tip lost	tip_missing	dose-not-delivered/tip_missing_or_dropped	tip_pick	yes	yes
tool changer slow	window_exceeded	prevents-violated	transport_lab, store	yes	yes
identity swap	identity_mismatch	identity-mismatch	quarantine	yes	yes
isaac calibration lapsed	calibration_invalid	measurement-invalid/calibration_expired	transport_lab, calibrate	yes	yes
isaac calibration lapsed	focus_invalid	measurement-invalid/out_of_focus	autofocus	NO	yes
isaac calibration lapsed	focus_invalid	measurement-invalid/out_of_focus	transport_lab, store	NO	yes

```
(meta-model epistemic
  (order (source observed reported inferred assumed predicted unknown))

  (rule quarantine-of-the-contradicted
    (when (a belief (of (the plate (identity verified))) (grade (corroboration contradicted)))
          (in (an action (type quarantining))))
    (treat proceed)
    (because "containing a plate whose identity is contradicted is the treatment, not something it may block"))
  (rule contradiction
    (when (a belief (grade (corroboration contradicted))))
    (treat contain-and-escalate)
    (because "two observations disagree; nothing that depends on the item may proceed"))
  (rule no-belief
    (when (a belief (grade (source unknown))))
    (treat sense-or-abstain)
    (because "no belief at all: sense if a sensor can, otherwise refuse the step"))
  (rule stale
    (when (a belief (grade (freshness stale))))
    (treat re-observe)
    (because "the observation predates a session boundary (kernel freshness rule F2)"))
  (rule prediction
    (when (a belief (grade (source predicted))))
    (treat plan-only)
    (because "a projection may guide planning, never a commitment"))
```

Listing 6: The first rules of the epistemic meta model (`registry/lab/epistemic.ldl`).

Table 18: Treatments for the plate-identity requirement of a dose, by shortfall of the belief.

Shortfall of the belief	Treatment	Rule
stale	re-observe	`stale`
contradicted	contain-and-escalate	`contradiction`
reported	verify-before-commit	`reported`
inferred	verify-if-sensable-else-escalate	`inferred-consequential`
assumed	verify-before-commit	`assumed-consequential`
predicted	plan-only	`prediction`
unknown	sense-or-abstain	`no-belief`

Table 19: All rules of the epistemic meta model.

Rule	When	Treatment	Because
`quarantine-of-the-contradicted`	(a belief (of (the plate (identity verified))) (grade (corroboration contradicted))); (in (an action (type quarantining)))	proceed	containing a plate whose identity is contradicted is the treatment, not something it may block
`contradiction`	(a belief (grade (corroboration contradicted)))	contain-and-escalate	two observations disagree; nothing that depends on the item may proceed
`no-belief`	(a belief (grade (source unknown)))	sense-or-abstain	no belief at all: sense if a sensor can, otherwise refuse the step
`stale`	(a belief (grade (freshness stale)))	re-observe	the observation predates a session boundary (kernel freshness rule F2)
`prediction`	(a belief (grade (source predicted)))	plan-only	a projection may guide planning, never a commitment
`reported-by-a-qualified-instrument`	(a belief (grade (source reported)) (from (an instrument (binding validated))))	proceed	a device report counts when the device's binding for it is validated
`reported`	(a belief (grade (source reported))); (use consequential)	verify-before-commit	an unqualified report is not enough for a consequential step
`assumed-consequential`	(a belief (grade (source (<= assumed)))); (use consequential)	verify-before-commit	a consequential step may not rest on a default
`weak-routine`	(a belief (grade (source (<= reported)))); (use routine)	proceed-flagged	routine use of a weaker belief is allowed and recorded
`inferred-consequential`	(a belief (grade (source (<= inferred)))); (use consequential)	verify-if-sensable-else-escalate	an inference is enough only where the contract says so

Table 20: KPIs of the experiment (`quality` clause).

KPI	Of	Aggregate	Target	Direction
safety	(each violation (of (a prevents-item (in ?E))))	count	<= 0	minimize
reliability	(each goal (of (an action (in ?E))))	first-attempt-fraction	>= 1	maximize
auditability	(each goal (of (an action (in ?E))))	evidenced-fraction	>= 1	maximize
excursion-time	(each excursion (of ?p))	max	<= 300 s	minimize
cost	(each resource (used-by ?E))	weighted-sum	<= 10 cost-units	minimize
dose-deviation	(each transfer (into (a well (of ?p))))	max	<= 2 %	minimize
time-to-result	?E	span	<= 72.5 h	minimize

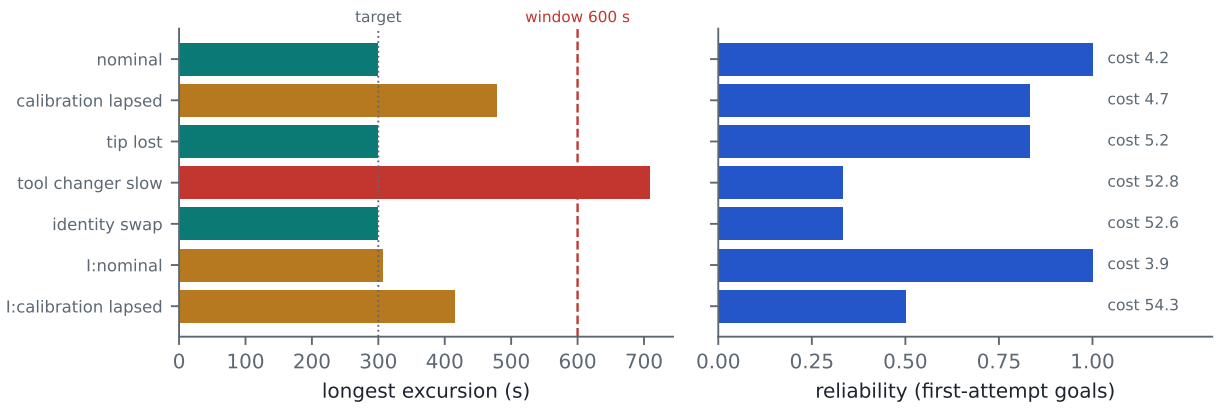


Figure 6: KPIs measured from the recorded trails: longest excursion outside the incubator and reliability; cost in the label.

Table 21: KPIs per recorded run, in the order of the lexicographic ranking.

Run	End	Safety	Reliability	Auditability	Max excursion (s)	Cost
nominal	executed	0	1.000	1.00	298.9	4.22
calibration lapsed	executed	0	0.833	1.00	478.7	4.69
tip lost	executed	0	0.833	1.00	298.9	5.24
tool changer slow	escalated	1	0.333	1.00	708.3	52.75
identity swap	escalated	0	0.333	1.00	298.9	52.61
isaac nominal	executed	0	1.000	1.00	305.7	3.89
isaac calibration lapsed	escalated	0	0.500	1.00	414.6	54.28

```
(the experiment ?E
  (question (which (a recovery-model (in (M1_graded M2_threshold M3_protective)))
                   (explains (the rescue-efficacy (after (a delay (of 48 h)))))))
  (entities
    (the culture ?cr (role rescued))
    (the culture ?cc (role comparator))
    (the culture ?ck (role control))
    (some substance ?ch (type challenge))
    (some substance ?rs (type rescue))
    (the belief ?B (over (M1_graded M2_threshold M3_protective)) (prior uniform)))
  ;; the design guarantees the gain in expectation (e2m); what one execution yields is measured, not promised
  (goal (the belief ?B (expected-information-gain (>= 0.5 bit))))
  (given
    (the culture ?cr (state baseline))
    (the culture ?cc (state baseline))
    (the culture ?ck (state baseline))
    (the belief ?B (entropy 1.585 bit))
    (the experiment ?E (culture-time 0 h)))
  (prevents
    (an exposure (of ?ck) (to (some substance)))           ; the control stays untreated
    (an event (type contamination) (of (a culture))))
  (maintains
    (the culture ?cr (environment culture-standard))
    (the culture ?cc (environment culture-standard))
    (the culture ?ck (environment culture-standard)))
  (steps
```

Listing 7: Head of the culture plan (`registry/lab/culture_plan.ldl`).

never exposed; no contamination) and what must hold throughout (the cultures' environment). Its steps are what is done to the cultures and what becomes known: challenge, grow, rescue, observe, interpret and analyse. Each step has a physical and a knowledge effect.

8.1 Derived order

The order is derived from 13 causal links, 7 physical and 6 knowledge links (Table 23); Fig. 7 shows the partial order. One link needs an establishable requirement: interpretation needs a known image scale. It first required "a calibration of the pixel size", which the independence check rejected because calibration is a device function (Section 13).

```
(an action ?c3
    (type rescuing)
    (culture ?cr)
    (substance ?rs)
    (dose 0.70 au)
    (at (culture-time 48 h))
    (requires (the experiment ?E (culture-time 48 h)))
    (physical (the culture ?cr (exposed-to ?rs 0.70 au))
              (the culture ?cr (recovery possible)))
    (knowledge (a record (of ?c3) (states ?cr ?rs 0.70 au 48 h))))
```

Listing 8: A culture-level action: the rescue.

Table 22: Steps of the culture plan with their physical and knowledge effects.

Step	Type	Physical effect	Knowledge effect
?c1	challenging	`(the culture ?cr (exposed-to ?ch 0.85 au)); (the culture ?cr (injury rising))`	`(a record (of ?c1) (states ?cr ?ch 0.85 au 0 h))`
?c2	challenging	`(the culture ?cc (exposed-to ?ch 0.85 au)); (the culture ?cc (injury rising))`	`(a record (of ?c2) (states ?cc ?ch 0.85 au 0 h))`
?g1	growing	`(the experiment ?E (culture-time 48 h)); (the culture ?cr (injury-burden accumulated))`	`(a record (of ?g1) (environment-history culture-standard))`
?c3	rescuing	`(the culture ?cr (exposed-to ?rs 0.70 au)); (the culture ?cr (recovery possible))`	`(a record (of ?c3) (states ?cr ?rs 0.70 au 48 h))`
?g2	growing	`(the experiment ?E (culture-time 72 h)); (the culture ?cr (viability settled))`	`(a record (of ?g2) (environment-history culture-standard))`
?o1	observing	`(the culture ?cr (light-dose increased))`	`(an image ?y1 (of ?cr) (culture-time 72 h) (raw true))`
?o2	observing	`(the culture ?cc (light-dose increased))`	`(an image ?y2 (of ?cc) (culture-time 72 h) (raw true))`
?o3	observing	`(the culture ?ck (light-dose increased))`	`(an image ?y3 (of ?ck) (culture-time 72 h) (raw true))`
?i1	interpreting	–	`(an estimate (of ?cr) (organoid-radius) (sigma 2.5 um))`
?i2	interpreting	–	`(an estimate (of ?cc) (organoid-radius) (sigma 2.5 um))`
?i3	interpreting	–	`(an estimate (of ?ck) (organoid-radius) (sigma 2.5 um))`
?a1	analysing	–	`(the belief ?B (posterior updated)); (the belief ?B (information-gain measured))`

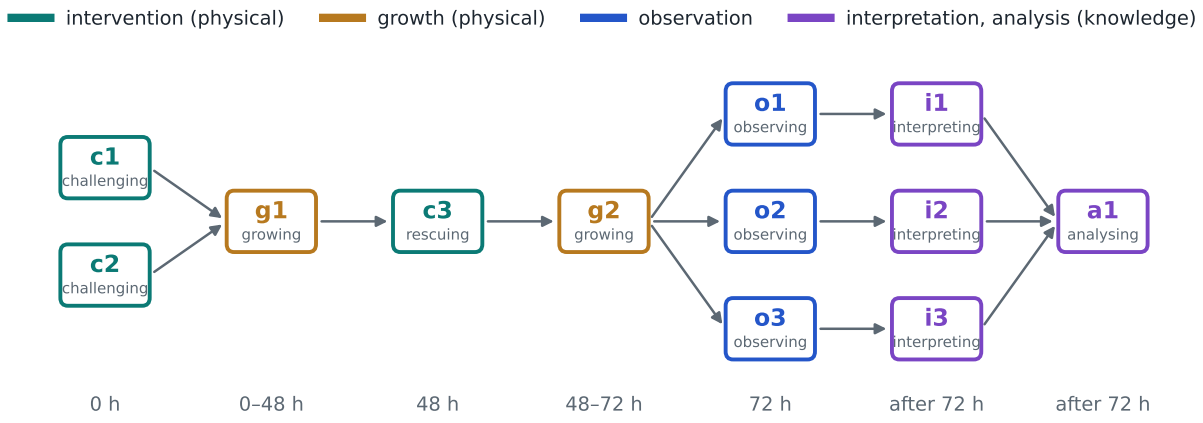


Figure 7: The culture plan as a partial order (Hasse layers). Colour shows the kind of step.

Table 23: Causal links of the culture plan.

From	To	Kind	Condition
?c1	?g1	physical	`(the culture ?cr (exposed-to ?ch))`
?c2	?g1	physical	`(the culture ?cc (exposed-to ?ch))`
?g1	?c3	physical	`(the experiment ?E (culture-time 48 h))`
?c3	?g2	physical	`(the culture ?cr (exposed-to ?rs))`
?g2	?o1	physical	`(the experiment ?E (culture-time 72 h))`
?g2	?o2	physical	`(the experiment ?E (culture-time 72 h))`
?g2	?o3	physical	`(the experiment ?E (culture-time 72 h))`
?o1	?i1	knowledge	`(an image (of ?cr) (raw true))`
?o2	?i2	knowledge	`(an image (of ?cc) (raw true))`
?o3	?i3	knowledge	`(an image (of ?ck) (raw true))`
?i1	?a1	knowledge	`(an estimate (of ?cr) (organoid-radius))`
?i2	?a1	knowledge	`(an estimate (of ?cc) (organoid-radius))`
?i3	?a1	knowledge	`(an estimate (of ?ck) (organoid-radius))`

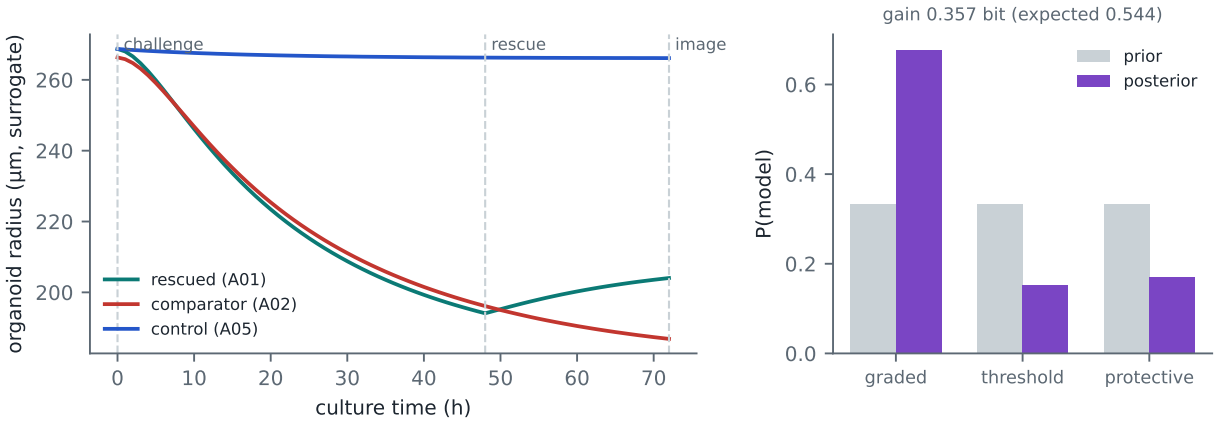


Figure 8: Surrogate organoid radii of the three cultures over culture time (left) and the belief over the recovery models before and after the experiment (right). ILLUSTRATIVE.

Table 24: What the experiment made known (surrogate, ILLUSTRATIVE).

Quantity	Value
observed radius A01 / A02 / A05 (µm)	201.6 / 186.8 / 267.6
measurement noise σ (µm)	2.49
observed rescue effect (µm)	14.76
predicted effect M1 / M2 / M3 (µm)	8.5 ± 11.4 / 25.0 ± 6.3 / 25.4 ± 6.5
posterior M1 / M2 / M3	0.678 / 0.152 / 0.170
entropy before / after (bit)	1.585 / 1.228
information gained (bit)	0.357
expected gain of the design / required (bit)	0.5439 / 0.5

8.2 Execution on the surrogate and what became known

Run on Cell 01's surrogate, the rescued culture recovers partly and the comparator does not (Fig. 8). The observed effect of 14.8 µm lies between the predictions of M1 and of M2/M3, so the belief moves towards M1 but gains only 0.357 bit, against 0.5 bit required and 0.5439 bit expected. The design meets the requirement only in expectation; the goal therefore says `expected-information-gain`. A priming design that separates M2 from M3 is on the roadmap.

8.3 Mapping to the laboratory

The cultures map to wells A01, A02 and A05 of plate P17; the eight physical culture actions map to lab action designators and, through their instrument roles, to L12, L10 and L17. The four computations (interpretation and analysis) map to the twin's image analysis and posterior computation. The lab order is exactly the projection of the culture order.

Feasibility. For each session the assistant checks every transport leg (both seats served by the robot, reach from the layout, seats typed in the SDT), the tool coupling, the machine tending of L10, L12 and L17, and the trip time against the window. All checks pass, with evidence from facts and from the Isaac run. They are conditional on two assumed device preconditions (L12 calibrated, L17 healthy), and inventory (tips, reagent stock) is not modelled.

9 Execution

The executive turns one linear extension of the lab plan, grouped by culture time into sessions, into a plan tree. The generalized plan of each category builds its sub-plan at perform time, querying the knowledge base; the leaves are motions and device commands. Before anything moves, the whole experiment is projected from the assistant's belief (F6, F12): every designator must be groundable, the understanding gate must accept every card, and the window policies predict instead of abort (F8). At each session start, observations of shared cell state go stale and are re-established (F2). If a session fails after the plate left the incubator, an unwind-protect returns it before the failure propagates, unless it was quarantined (F10).

The executive is also emitted as a CRAM/RPL structured reactive controller (409 lines of Lisp): one `def-action-plan` per lab action type, with the contract's requires as checks, the scoped prevents as monitors, and the handlers

Table 25: Refinement of the culture actions to lab actions, processes and devices.

Culture action	Lab action	Process	Realized by
?c1 challenging	?x1 intervening	substance-transfer	L12
?c2 challenging	?x2 intervening	substance-transfer	L12
?g1 growing	?w1 incubating	incubation	L10
?c3 rescuing	?x3 intervening	substance-transfer	L12
?g2 growing	?w2 incubating	incubation	L10
?o1 observing	?m1 measuring	imaging	L17
?o2 observing	?m2 measuring	imaging	L17
?o3 observing	?m3 measuring	imaging	L17
?i1 interpreting	computation		culture SDT image analysis (twin/culture/optics.analyse): radius from the calibrated pixel size
?i2 interpreting	computation		culture SDT image analysis (twin/culture/optics.analyse): radius from the calibrated pixel size
?i3 interpreting	computation		culture SDT image analysis (twin/culture/optics.analyse): radius from the calibrated pixel size
?a1 analysing	computation		posterior over M1-M3 (twin/culture/eig.posterior_over_models) on the rescued - comparator radius

Table 26: Instantiation of the abstract entities in Cell 01.

Abstract entity	Role	Particular in Cell 01
`?cr`	rescued	the organoid in P17:A01
`?cc`	comparator	the organoid in P17:A02
`?ck`	control	the organoid in P17:A05
`?ch`	challenge	reagent abstract_challenge_reagent (barcode RGT-0001, lot SIM-LOT-A) at l12_liquidhandler_reagent_rack_frames_rack_1_pos_1
`?rs`	rescue	reagent abstract_rescue_reagent (barcode RGT-0002, lot SIM-LOT-B) at l12_liquidhandler_reagent_rack_frames_rack_1_pos_2
`(the plate ?p)`	carrier	Plate_P17 in L10 cassette slot 2

as failure handling. Listing 9 shows the plan for `intervening`.

9.1 Cross-check with the planner

Expanding the nominal plan into designators reproduces the clingo plan's leaf steps term for term (153 leaves, same primitive, agent and pose, in the same order), and the durations match the planner's exactly (298.9, 263.3, 256.4 s). Two independent derivations from the same facts agree. Since the symbolic backend takes durations from the planner's facts, this shows consistency, not independent physics; the Isaac runs supply the latter.

9.2 Scenarios

Twelve symbolic scenarios exercise the kernel with the *same plan code*; only knowledge and world change (Table 28). Expectations, including the two-column verdict and the disposition, were written before the runs. They cover device exchange, step substitution by knowledge and by repair, workflow change, safety and identity, and knowledge gaps.

- **Device exchange.** `imager_swapped` executes unchanged and ends NR3: physically realised, invalid evidence, nothing committed. `imager_incapable` is refused before the first intervention, naming the missing capability.

Table 27: Feasibility checks per session.

Session	Culture time	Device	Transport legs	Checks	All pass
s0	0 h	L12	R01 L10→HANDOVERNEST; R02 HANDOVERNEST→L12; R02 L12→HANDOVERNEST; R01 HANDOVERNEST→L10	8	yes
s1	48 h	L12	R01 L10→HANDOVERNEST; R02 HANDOVERNEST→L12; R02 L12→HANDOVERNEST; R01 HANDOVERNEST→L10	8	yes
s2	72 h	L17	R01 L10→HANDOVERNEST; R02 HANDOVERNEST→L17; R02 L17→HANDOVERNEST; R01 HANDOVERNEST→L10	8	yes

```
(def-action-plan intervening ((into ?w) (substance ?s) (dose ?q) (at ?t))
  "Goal: (the well ?w (received ?s ?q)). Performed in: x1, x2, x3."
  (let* ((?self (current-action-designator))
         (?p (plate-of ?w))
         (?instrument (an instrument
                          (slot (a plate-slot (accepts ?p)))
                          (process (a process
                                      (type substance-transfer)
                                      (into ?w)
                                      (of ?s)
                                      (amount ?q))))))
    (with-retry-counters ((wrong-reagent-repairs 1) (tip-missing-or-dropped-repairs 1))
      (with-failure-handling
          ((dose-not-delivered (f)
             (case (failure-cause f)
               ;; perform again; guarantees goal-or-safe-state: holds
               (wrong_reagent (do-retry wrong-reagent-repairs
                                (achieve (some substance ?s (identity verified)))
                                (retry)))
               ;; perform again; guarantees goal-or-safe-state: holds
               (tip_missing_or_dropped (do-retry tip-missing-or-dropped-repairs (retry))))))
        (seq
          (ensure (the plate ?p (identity verified))
                  :grade (freshness fresh source observed)
                  :consequential t)
          (ensure (some substance ?s (identity verified))
                  :grade (freshness fresh source observed)
                  :consequential t)
          (ensure (the plate ?p (contamination none))
                  :grade (freshness fresh source inferred)
                  :consequential t)
          ;; supplied by a predecessor in the partial order
          (wait-for (holds (the plate ?p (culture-time ?t))))
          (with-policy (par
                         (prevent (holds (an event (type contamination-introduced) (of ?p)))
                                  contamination-suspected)
                         (scope ((holds (a portion ?u (of ?s) (drawn-for ?self)))
                                 (holds (the portion ?u (delivered true))))
                           (prevent (holds (a transfer (of ?u) (into (a well (of ?p) (not ?w)))))
                                    prevents-violated)))
            (perform (an action (type tending) (instrument ?instrument) (object ?p))))
          (check-goal (the well ?w (received ?s ?q)) dose-not-delivered))))))
```

Listing 9: The generated CRAM/RPL plan for the lab action type `intervening`.

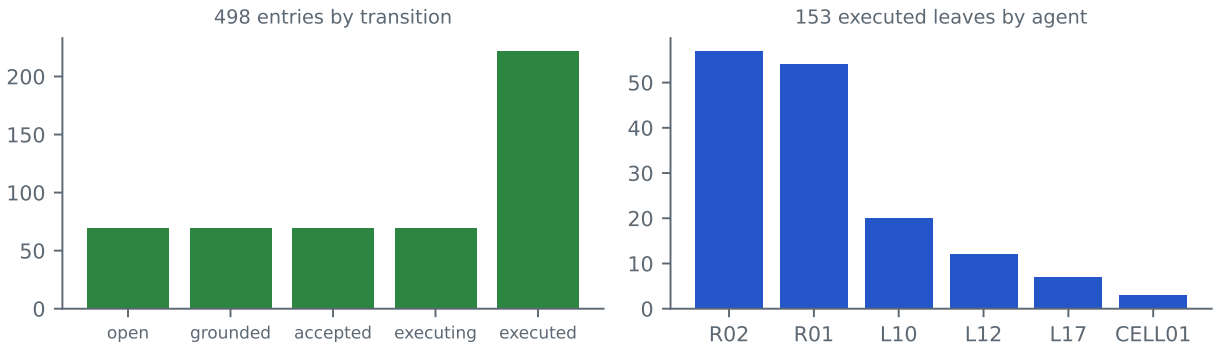


Figure 9: The nominal audit trail: entries by lifecycle transition and executed leaves by agent.

Table 28: Scenario catalogue: what is injected and how each run ends.

Scenario	Fault or change injected	Outcome
nominal	the derived 0.5-bit experiment, fault-free	executed
calibration lapsed	the KB believes L17 calibrated but it lapsed; detected at imaging, repaired by QC and re-imaging	executed
card incomplete	the dose category card lacks its evidence obligations; the understanding gate refuses the dose	abstained: refused: refused by projection_succeeds
contamination	contamination is suspected; challenge and rescue are refused	abstained: refused: refused by projection_succeeds, contamination_free
control dropped	the workflow is changed to drop the control well; the experiment is refused before any motion	abstained: refused: refused by projection_succeeds, challenge_requires_controls
identity swap	L10 holds P18 instead of P17 at the rescue session; quarantine P18, escalate	escalated: escalated: l10 read P18, expected P17 (after quarantine P18 (Cell 01 routing H4_quarantine_if_ identity_unresolved); P17 location…
imager incapable	L17 replaced by a device without well imaging; the measurement session cannot be grounded	abstained: refused: refused by projection_succeeds
imager swapped	L17 replaced by a capability-matched imager without a validated binding for the readout (device exchange)	executed
rescue unbound	the rescue reagent is not bound to a rack position; the rescue dose cannot be grounded	abstained: refused: refused by projection_succeeds
tip lost	the tip falls off after the pick at the rescue dose; detected before aspiration, tip replaced	executed
tool changer slow	the tool changer takes 600 s at the rescue session; window exceeded, safe return	escalated: session_failed: policy window: 645.3 s outside the incubator (window 600.0 s) (after safe return of the plate to its controlled …
uncalibrated known	the KB does not assert calibrated(l17); a QC calibration is planned before imaging (step insertion by knowledge)	executed
isaac nominal	the derived 0.5-bit experiment on the Cell 01 twin	executed
isaac calibration lapsed	L17's calibration is invalidated just before the first image of s2; set aside, recalibrate at L21, re-plan	escalated: session_failed: A02 out of focus; retained (after safe return of the plate to its controlled environment (focus_invalid in measu…
isaac identity swap	the experiment is about P18 (L10 inventory slot 5) but the plate in slot 5 is P23 (twin initial state); L10's barcode read catches it, P23 is quarantined in H4, escalation	escalated: escalated: l10 read P23, expected P18 (after quarantine P23 (Cell 01 routing H4_quarantine_if_ identity_unresolved); P18 location…

Table 29: Outcome per scenario: end state, realization (R) and validity (V), repairs, trail size, and whether it ended as specified.

Scenario	Backend	End	R | V	Repairs	Entries	As specified
nominal	symbolic	executed	ok | ok	–	498	yes
calibration lapsed	symbolic	executed	ok | ok	image	569	yes
card incomplete	symbolic	abstained	– | –	–	3	yes
contamination	symbolic	abstained	– | –	–	3	yes
control dropped	symbolic	abstained	– | –	–	3	yes
identity swap	symbolic	escalated	– | –	retrieve	224	yes
imager incapable	symbolic	abstained	– | –	–	3	yes
imager swapped	symbolic	executed	ok | –	–	498	yes
rescue unbound	symbolic	abstained	– | –	–	3	yes
tip lost	symbolic	executed	ok | ok	dose	501	yes
tool changer slow	symbolic	escalated	– | –	run_session	266	yes
uncalibrated known	symbolic	executed	ok | ok	–	504	yes
nominal	Isaac Sim	executed	ok | ok	–	498	yes
calibration lapsed	Isaac Sim	escalated	– | –	run_session, image, image	562	NO
identity swap	Isaac Sim	escalated	– | –	retrieve	56	yes

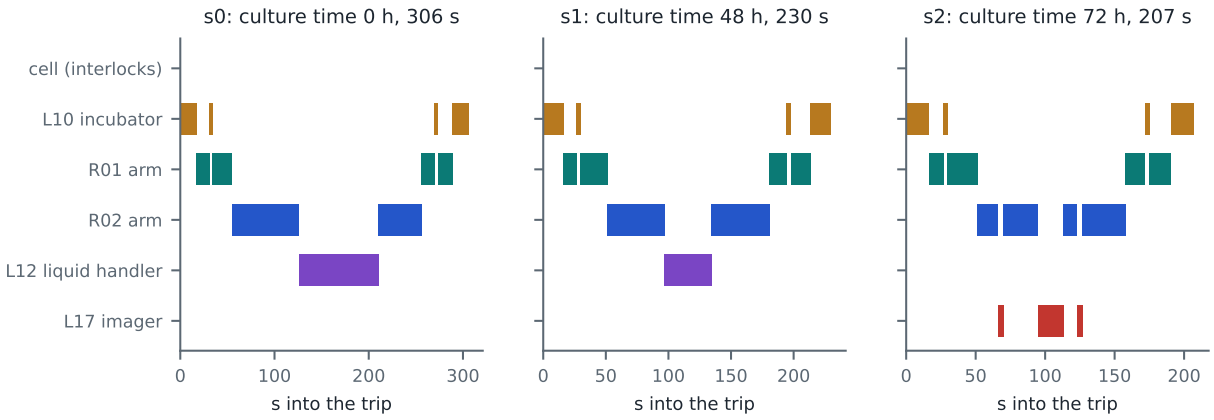


Figure 10: The nominal run in Isaac Sim: leaves per agent over each session.

- **Step substitution.** `uncalibrated_known` inserts a QC calibration as a preparation step. `calibration_lapsed` detects the lapse at imaging, sets the plate aside at a passive station, recalibrates and re-plans the measurement. `tip_lost` replaces the tip before aspiration.
- **Workflow change.** `control_dropped` is refused by `challenge_requires_controls` before anything moves.
- **Safety and identity.** `identity_swap` quarantines the wrong plate, escalates, and records that it does not know where P17 is. `tool_changer_slow` exceeds the window and returns the plate before reporting failure.
- **Knowledge gaps.** `rescue_unbound`, `contamination` and `card_incomplete` are refused before any motion.

9.3 Isaac Sim execution

The Isaac backend runs the same plan code, knowledge, audit, critic and competency questions; only leaf execution changes. The twin's skills are coarser than the leaves (F13): a robot pick is approach, insert, grasp and lift; L12's `dose_well` is tip pick, aspirate, dispense and tip eject. A group of leaves is executed at its first leaf, and the following leaves report the phases of that execution; every leaf entry records what realised it. Durations are physics time; culture waits advance a separate clock.

The twin's session durations deviate from the planner's by +2.3 %, -12.6 %, -19.1 % and stay within the window. `isaac_identity_swap` ends as specified: L10 read the wrong barcode, the plate was quarantined at H4 and the run escalated.

Finding F17: `isaac_calibration_lapsed` does not end as specified. The calibration repair worked: L17 refused, the plate was set aside, L21 recalibrated and the measurement was re-planned; A01 was then imaged validly. A02 then failed the focus floor twice (scores 0.035 and 0.030, floor 0.04); the session returned the plate and escalated, so control A05 was never imaged. After the reload, the A01 field is shifted by about 93 µm against the nominal run with the same commanded stage position and seat sensors at zero gap; two A02 images of the same load differ by about 35 µm. The plate seems to move in the nest, which the seat sensors do not see. The kernel's repair `focus_invalid → autofocus, retry` cannot change the outcome, because L17 autofocuses before every image; the process model prescribes a re-seat, and the LDL checker catches the refocus-only handler (N9). The disposition was correct for what happened; the expected outcome was not reached, and the expectation was not changed.

10 Audit trail, critic and competency questions

Every lifecycle transition of every designator writes one commitment record: sequence number, simulated time, designator and parent, kind and category, transition, agent and software versions, the HADL term, bindings, resolution grounds, constraint verdicts, monitor results and evidence references, each with its own SHA-256. Each entry carries the hash of its predecessor and its own hash over the canonical JSON, forming a chain [17]. A verifier recomputes the chain; editing any value breaks it at that entry. Listing 10 shows a dispense entry.

Critic. The critic judges each episode from the trail alone, in two columns. Realization: the run ended executed and no runtime constraint was violated. Validity: V1–V7 (identity continuity, calibration at measurement, validated binding, controls measured, dose delivered, sample integrity, design realised). The result may update the culture belief only if both hold (D009).

```
{
  "seq": 86,
  "t_sim": 156.1,
  "designator": "d0039-dispense",
  "parent": "d0036-dose",
  "kind": "command",
  "category": "dispense",
  "transition": "executed",
  "agent": "l12",
  "software": {"kernel": "rsa-kernel/0.1", "kb": "facts_g050+nominal", "registry": "5d901b7f8087...
  "term": ["cmd", "l12", ["dispense", "https___aicor_example_okf_sdt_lab1_world_experimentalworl...
  "monitors": [{"name": "well_binding", "passed": true, "observation": "P17/A01"}, {"name": "dis...
  "constraints": [{"id": "l12_airspace_clear", "column": "realization", "verdict": "ENFORCED", "...
  "evidence": [  // 10 items, each with its own sha256
    {"kind": "plate", "value": "P17", "sha256": "6f418d76902ea3669355d6f0f3abf4387fe78c83d8b...,
    {"kind": "well", "value": "A01", "sha256": "3b1d4717064134258d9fc1cdb2a8055c242348548558...,
    {"kind": "reagent", "value": "challenge", "sha256": "d0303e386ced87e527d925fae2b47bbbf40...,
    ...],
  "prev": "32dddeae06c3eed2acaa64bf9e9f446a834a8539ff4e35f1ec28bf0ded3ce3b9",
  "hash": "e890f45744b3ada383becbcd869882c65acd04239017d16b015f077ee6d9c739",
}
```


Listing 10: An audit entry of the nominal trail (a dispense), abbreviated.

Table 30: Competency-question checks by family.

Family	Checks	Correct
what	3	3
why	4	4
how	2	2
constraints	3	3
capability	4	4
what-if	3	3
evidence	4	4
retrospective	13	13
normative	4	4
verdict	8	8
epistemic	2	2
taxonomy	1	1
modularity	1	1

10.1 Competency questions

27 competency questions in 13 families ask what the assistant is doing, why, how, under which constraints, with which capability, what would happen if, with what evidence, what went wrong, which norms are covered, whether the result is valid, what it does not know, what kind of action something is, and what a device exchange would change. They are answered from the trail, the knowledge base and dry runs, with grounds. Expected answers were written by hand for selected scenarios before the runs: 52 checks, 52 correct and all grounded.

10.2 The kernel in the ontology

The kernel's categories and the trail are exported to an OKF module. Each category is a class defined by its level and kernel name, so trail individuals are classified by the reasoner; effect classes and facets are disjoint superclasses aligned with the e2m designator classes. HermiT finds no unsatisfiable class after F14; Konclude realises 6244 individuals, and all 222 designators and 498 audit entries of the nominal trail are classified with none left over. 18 TBox competency questions and 6 negative controls pass.

11 Motion level

Every motion leaf of the executed plan is refined into a Giskard motion designator: 111 leaves become 111 designators over 12 legs, 6 of them distinct. Each designator has reference frames (robot root and tip, and a goal pose in a seat frame of the composed scene), motion constraints (Cartesian goal or straight line, speed limits, plate kept level,

```
{
 "id": "CQ-03",
 "scenario": "nominal",
 "question": "Why was this device chosen for dosing, and which alternatives were rejected?",
 "answer": {
  "device": "l12",
  "grounds": [
   "automated(l12)",
   "has_function(l12,dispensing)",
   "component:L12_LiquidHandler.capabilities in dose_well"
  ],
  "rejected": []
 },
 "grounds": [
  "trail#81:dose/grounded",
  "automated(l12)",
  "has_function(l12,dispensing)",
  "component:L12_LiquidHandler.capabilities in dose_well"
 ]
}
```


Listing 11: An answer with grounds: why L12 was chosen for dosing and which devices were rejected (CQ-03, nominal).

Table 31: Distinct transport legs and their motion sequences.

Robot	Leg	Motions	Sequence
R01	L10 → HANDOVERNEST	9	approaching → inserting → grasping → lifting → transiting → aligning → placing → releasing → retracting
R02	HANDOVERNEST → L12	10	coupling → approaching → inserting → grasping → lifting → transiting → aligning → placing → releasing → retracting
R02	L12 → HANDOVERNEST	9	approaching → inserting → grasping → lifting → transiting → aligning → placing → releasing → retracting
R01	HANDOVERNEST → L10	9	approaching → inserting → grasping → lifting → transiting → aligning → placing → releasing → retracting
R02	HANDOVERNEST → L17	10	coupling → approaching → inserting → grasping → lifting → transiting → aligning → placing → releasing → retracting
R02	L17 → HANDOVERNEST	9	approaching → inserting → grasping → lifting → transiting → aligning → placing → releasing → retracting

collision avoidance, allowed contacts), an ending monitor and cancel conditions, and the objective of giskardpy's QP-based model-predictive controller. The output is a Lisp description and a giskardpy module of 1958 lines whose API use was checked against a CRAM checkout.

All 25 goal frames are reachable with the FR3 and the rail-mounted UR5e kinematics within joint-margin and conditioning thresholds (Fig. 11). R01's goals are collision-free; R02's collisions are not checked offline. Compliant insertion is open: giskardpy has no impedance control, and a force monitor needs ROS.

Table 32: Controller parameters and motion bounds.

Parameter	Value	Meaning
control frequency	50 Hz (dt 0.02 s)	QP-based MPC rate
prediction horizon	7	steps; highest derivative jerk; solver PIQP
weights	level 2500, speed 2500, collision 50, goal 1, posture 0.1	level and speed dominate; goal tracking is soft
plate tilt	≤ 2.0°	kept level while held (cancel if violated)
transit speed	≤ 0.25 m/s, 0.6 rad/s	Cartesian limits while carrying
pose tolerance	0.4 mm, 0.2°	end monitor at grasp / release (A3)
lock gap	4 mm	nest capture before release
timeout	1.2 × estimate	the ±20 % duration bound of z3 theorem V3

```
def inserting(world, robot, seat, z, speed, tol, plate):
    root, tcp = body(world, robot["root"]), body(world, robot["tcp"])
    goal = tool_down(body(world, seat), z)
    done = PoseReached(root_link=root, tip_link=tcp, goal_pose=goal.to_homogeneous_matrix(),
                       position_threshold=tol, orientation_threshold=ORI_TOL)
    aux = [CartesianPoseStraight(root_link=root, tip_link=tcp, goal_pose=goal),
           CartesianPositionVelocityLimit(root_link=root, tip_link=tcp, max_linear_velocity=speed,
           ↪  weight=W_SPEED),
           allow_contact([body(world, j) for j in robot["finger_links"]], [plate])]
    return done, aux


def straight(world, robot, seat, z, speed, plate=None, contact=None):
    """lifting, placing, retracting: straight vertical motion of the TCP at a capped speed."""
    root, tcp = body(world, robot["root"]), body(world, robot["tcp"])
    done = CartesianPositionStraight(root_link=root, tip_link=tcp,
                                     goal_point=Point3(0.0, 0.0, z, reference_frame=body(world, seat)),
                                     reference_velocity=speed, threshold=POSE_TOL)
    aux = [CartesianPositionVelocityLimit(root_link=root, tip_link=tcp, max_linear_velocity=speed,
    ↪  weight=W_SPEED)]
    if plate is not None:
        aux.append(keep_level(world, plate))
    if contact is not None:
        aux.append(allow_contact(*contact))
    return done, aux
```

Listing 12: The generated giskardpy statechart for inserting into a seat.

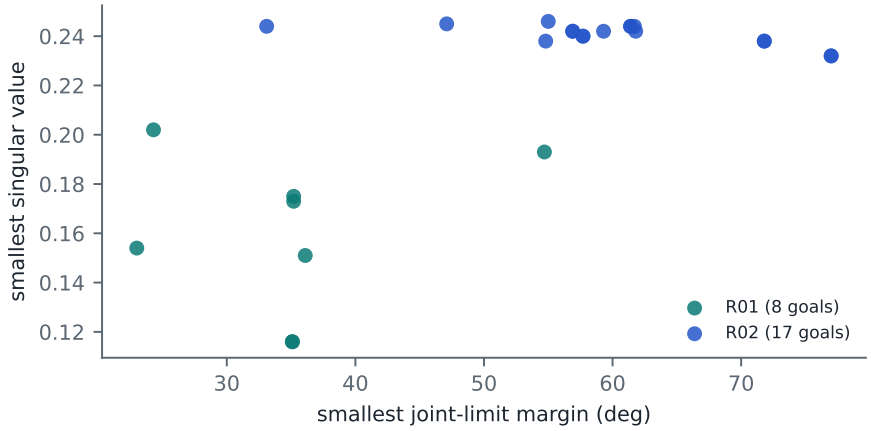


Figure 11: Reachability of the goal frames: joint-limit margin and smallest singular value of the Jacobian per goal.

12 Refinement proof

The refinement proof shows that the motion-level plan executes the culture-level plan and satisfies the stated constraints, relative to explicit assumptions about controllers, sensors and devices. It is an assume-guarantee argument in the sense of refinement mappings and contract-based design [18, 19].

Theorem. Let Π_M be the motion-level plan of the experiment: the 151 grounded steps whose 111 robot motions are refined into Giskard statecharts. Under A1–A6, every execution of Π_M either (i) realizes the culture plan: every physical and knowledge effect in the derived order at the design's culture times, with the control untreated, while every stated constraint holds on the way (interlocks, L12 airspace, identity, environment before store, calibration before imaging, the window, contamination freedom, tool hygiene, plate level, collision distance); or (ii) is stopped by a monitor and ends in a failure handler whose guarantee holds (safe state, contained, or the goal).

Motion constraints are hold-or-cancel: a statechart ends only through its end monitor or a cancel, and every cancel (timeout at 1.2 × the estimate, tilt, collision distance) reaches a handler with a verified guarantee (O7). The proof does not cover progress (that a motion terminates), the two partly detected faults, contact physics, or Giskard's execution itself.

Table 33: Assumptions of the refinement proof.

	Assumption	Covered by
A1	a motion statechart ends only through EndMotion (its end monitor true) or CancelMotion.	library semantics
A2	PoseReached, LocalMinimumReached with the grasp check, VectorsAligned, the collision distance monitors, the simulation clock, the barcode read, the seat sensors and the device monitors observe correctly.	assumed; 2 fault kinds only partly detected
A3	a TCP within 0.4 mm / 0.2 deg of a seat's grasp or release pose has the plate in that seat (the nest guides capture it); within 5 mm of an approach pose it is at that approach pose (free-space waypoint).	ILLUSTRATIVE capture (2 mm)
A4	grasp_confirmed, lifted, seating).	monitored at run time
A5	dose_well delivers within its tolerance when its monitors pass; image_well yields an image when calibration and focus monitors pass; L10 holds the environment profile while the plate is stored.	component contracts, monitored
A6	the goal frames are the ones the scene defines (rsa/motion_frames.py), and each is reachable (checked).	checked (25/25)

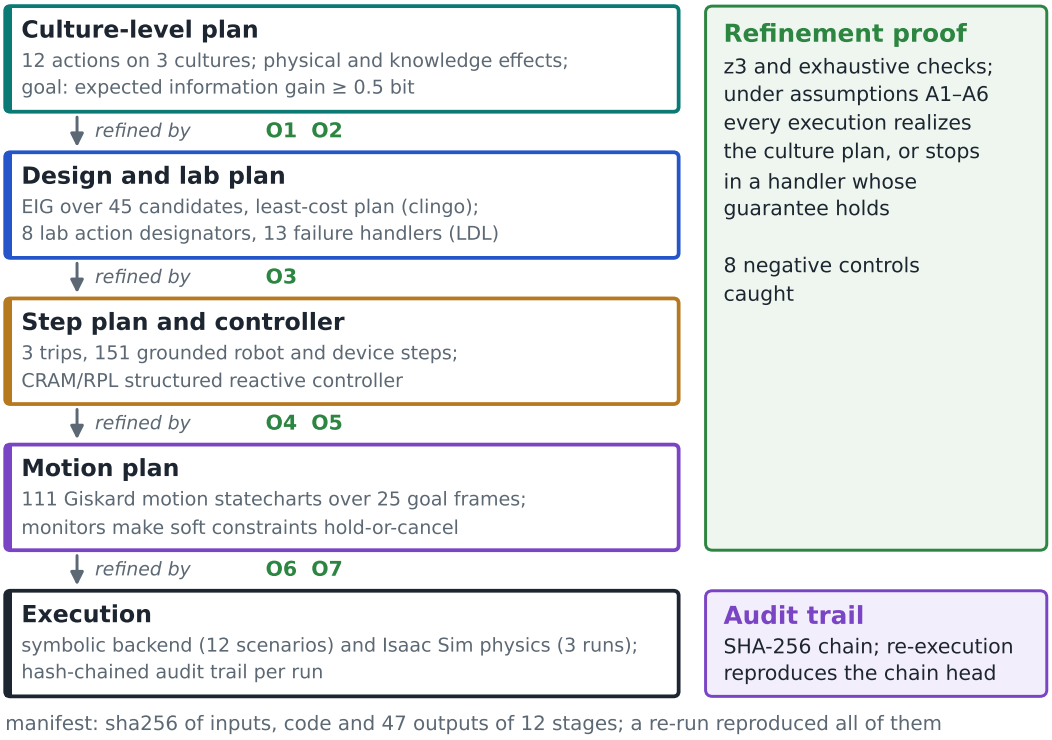


Figure 12: The proof chain: levels, the obligation discharged between each pair, and the audit.

Table 34: Proof obligations and how they are discharged.

	Obligation	Discharged by	Result
O1	culture plan well formed (supported, acyclic)	causal-link analysis	13 links
O2	step-level events entail every culture effect, the order and the culture prevents	z3 (negation UNSAT)	14/14
O3	lab plan: links, threats, instruments, handler guarantees	LDL checker	threats 9/9, guarantees 13/13
O4	step plan: executability, safety, time, realization, gain, least cost	z3 (V1–V5)	7/7 theorems
O5	no plate-handling tool touches the culture	z3	holds
O6	each motion refines its step; end monitor implies the postcondition; soft parts hold-or-cancel	exhaustive check	111/111 motions
O7	every cancellation reaches a handler with a verified guarantee	exhaustive check	13 failure types

Table 35: Obligation O2: every culture-level effect is entailed by step-level events (z3).

O2 obligation (culture-level effect entailed by step-level events)	Proved
?c1 challenging ?cr: exposed to challenge 0.85 au at 0 h	yes
?c2 challenging ?cc: exposed to challenge 0.85 au at 0 h	yes
?g1 growing 0 -> 48 h: cultures in their environment between the trips	yes
?c3 rescuing ?cr: exposed to rescue 0.70 au at 48 h	yes
?g2 growing 48 -> 72 h: cultures in their environment between the trips	yes
?o1 observing ?cr: image at 72 h	yes
?o2 observing ?cc: image at 72 h	yes
?o3 observing ?ck: image at 72 h	yes
?i1 interpreting ?cr: image and known scale	yes
?i2 interpreting ?cc: image and known scale	yes
?i3 interpreting ?ck: image and known scale	yes
?a1 analysing: estimates of the rescued, comparator and control cultures	yes
prevents: the control culture is never exposed	yes
order: challenges at 0 h < rescue at 48 h < observations at 72 h	yes

Table 36: Negative controls of the refinement proof.

Mutation	Caught
vessel map swaps rescued and comparator	yes
the control well A05 is also dosed	yes
the control well is not imaged	yes
the plate tool T01 touches the culture	yes
one held motion without the tilt cancel	yes
one motion's timeout beyond V3's bound (1.5 x)	yes
a motion that does not refine its step	yes
step level: the validator's own 10 mutated plans	yes

13 Independence of the culture plan

The culture plan must not depend on the laboratory. Two checks establish this. **Vocabulary:** no term of six realization vocabularies (robots, devices, lab setup, safety constraints, cleanliness, temperature control), collected from the facts, the component records and the registries, occurs in the culture plan. The check found one leak, a requirement for "a calibration of the pixel size"; it now requires a known image scale. **Invariance:** in 13 laboratory variants the culture plan, its order and the lab plan are identical; only the realization differs, and some laboratories refuse before anything moves (Table 37).

Table 37: The culture plan in 13 laboratory variants. The culture-plan hash is the same in all.

Lab variant	Dimension	Outcome	Culture plan
Cell 01 as built (layout P1)	(reference)	realized	a5e27614c98a
one rail robot serves every station (no R01, no handover)	robots	realized	a5e27614c98a
R02 keeps its plate tool coupled between trips	robots	realized	a5e27614c98a
R01 and R02 hand the plate over in the plate hotel L09, not a handover nest	lab setup	realized	a5e27614c98a
the imager replaced by a capability-matched model without a validated binding	devices	realized, result not valid evidence (NR3)	a5e27614c98a
the imager replaced by another model with a validated binding for the readout	devices	realized	a5e27614c98a
the liquid handler replaced by another model with the same capabilities	devices	realized	a5e27614c98a
the lab allows the plate only 300 s outside the incubator	safety constraints	refused before anything moved	a5e27614c98a
the enclosure interlocks are open	safety constraints	refused before anything moved	a5e27614c98a
contamination is suspected on the plate	cleanliness	refused before anything moved	a5e27614c98a
the lab classes the plate tool T01 as touching the culture	cleanliness	refused before anything moved	a5e27614c98a
the incubator keeps a different environment profile than culture-standard	temperature control	refused at mapping	a5e27614c98a
the imager's calibration is not current	calibration	realized	a5e27614c98a

Table 38: Stages of the reproducible derivation and their reproduction.

Stage	Derives	Outputs	Identical
design	EIG over the competing recovery models; least-cost safe plan meeting 0.5 bit (clingo)	4	4
step-proof	SMT validation of the plan: executability, safety, time, realization, gain, least cost (z3)	1	1
culture	culture plan: causal order, surrogate execution, mapping to the lab, transport and tending feasibility	1	1
lab-plan	lab action designators: order, threats, instruments, process models, handlers, guarantees, KPIs	1	1
controller	CRAM/RPL structured reactive controller (Lisp)	3	3
execution	kernel execution on the symbolic backend: 12 scenarios, audit trails, critic, tamper test, competency questions	26	26
grounding	cultures on plates, trips, device programs, monitors, reactions	1	1
frames	goal frames from the scene; reachability (IK) of every goal	1	1
motions	Giskard motion statecharts: frames, constraints, monitors, cancels	3	3
independence	the culture plan in 13 lab variants	1	1
refinement-proof	motion plan refines the culture plan (O1-O7, negative controls)	2	2
protocol	experimentation plan (protocol) and audit certificate: chains, tamper test, re-execution of every symbolic trail	3	3

Table 39: Recorded inputs of the derivation.

Input group	Files	Digest (first 12)
culture plan and lab designators (LDL)	lab	96cd481e0c09
kernel registry (categories, constraints, devices, norms, scenarios, competency questions)	categories.yaml, constraints.yaml, devices.yaml, norms.yaml, scenarios.yaml, cq.yaml	913f6adfbb00, 8de3c640bb68, e6a148880ab5, bdeceb9ea0c6, 2eb7c6867b0d, 695c0890a0dc
lab ontology ABox (semantic digital twin, safety, experiments)	sdt.ofn, safety.ofn, experiments.ofn	ad6743c73784, ed3d2a9a452f, 04bb8afcc391
experiment-to-motion axioms (ASP)	experiment_to_motion.lp	0fc82df2f9b6
Cell 01 scene (composed overlay)	cell01_twin.usda	6f1aa9b4f182
culture surrogate (ILLUSTRATIVE, D016)	culture	eb16756e1583
twin duration traces	cell01_episodes_raw.json	306cad4baa5d
recorded Isaac Sim runs (physics; read, not re-executed, by this pipeline)	trail_isaac_nominal.json, trail_isaac_identity_swap.json, trail_isaac_calibration_lapsed.json	583d3b3f201f, 1f700253c8b3, b3938e6fc36a

14 Reproducible derivation, plan and audit

The derivation runs in 12 stages, from the EIG design to the protocol. A manifest records the hash of every input, the code each stage ran, every output and the tool versions; the manifest's own hash identifies the whole derivation. JSON outputs are hashed in canonical form without wall-clock fields; nothing else is excluded. `--verify` re-runs every stage, including a fresh EIG Monte Carlo, and compares: all 47 outputs were identical and the manifest hash `a8a39851a44328aa` was reproduced.

Experimentation plan. `rsa/out/repro/protocol.md` is generated from the outputs: question and design with alternatives, culture plan and order, culture-to-well mapping, refinement into lab actions and devices, trips, constraints, handlers, KPIs, acceptance criteria and the records, bound to the derivation by an input digest. Appendix Section A reproduces its tables.

Audit certificate. For all 15 trails, the chain is intact and a single changed value is detected. Each of the 12 symbolic scenarios was re-executed in a temporary directory and produced the identical trail, chain head included

Table 40: Tool versions recorded in the manifest.

Tool	Version
python	3.12.3
clingo	5.8.2
z3	5.3.1
numpy	2.5.4
platform	Linux-7.0.0-34-generic-x86_64-with-glibc2.39
usd	pxr

Table 41: Audit certificate: chain, tamper test and re-execution per trail.

Trail	Entries	Evidence	Chain	Tamper caught	Re-executed	End	Disposition
calibration lapsed	569	51	intact	yes	identical	executed	commit: result may update the culture belief
card incomplete	3	0	intact	yes	identical	abstained	refused before execution
contamination	3	0	intact	yes	identical	abstained	refused before execution
control dropped	3	0	intact	yes	identical	abstained	refused before execution
identity swap	224	29	intact	yes	identical	escalated	escalated: human decision required; nothing…
imager incapable	3	0	intact	yes	identical	abstained	refused before execution
imager swapped	498	50	intact	yes	identical	executed	NR3: physically realised but not valid evid…
isaac calibration lapsed	562	219	intact	yes	not re-run	escalated	escalated: human decision required; nothing…
isaac identity swap	56	15	intact	yes	not re-run	escalated	escalated: human decision required; nothing…
isaac nominal	498	200	intact	yes	not re-run	executed	commit: result may update the culture belief
nominal	498	50	intact	yes	identical	executed	commit: result may update the culture belief
rescue unbound	3	0	intact	yes	identical	abstained	refused before execution
tip lost	501	51	intact	yes	identical	executed	commit: result may update the culture belief
tool changer slow	266	29	intact	yes	identical	escalated	escalated: human decision required; nothing…
uncalibrated known	504	51	intact	yes	identical	executed	commit: result may update the culture belief

(Table 41). The Isaac trails are verified but not re-executed; GPU physics is not expected to be bit-reproducible, and they were recorded under older registry hashes.

15 Findings log

Each iteration ended with findings that changed the kernel, the knowledge or the language. The table keeps the ones that changed concepts; the expectation tables were not changed after the runs, except two disclosed corrections in iteration 1 (repairs are recorded on the node that repaired; `session_failed` routes as escalation).

What the reconstruction learned. A device-command designator is needed beside motions. Prospection operates at two levels (experiment and session) and must predict policies, not enforce them. Belief needs freshness. The understanding gate is cheap and belongs in prospection. Physical success and validity separate naturally once the measurement binding is a knowledge item. And repairs must be justified by the device's process model: a repair that repeats the same deterministic action cannot help.

16 Limitations and open questions

- **No wet laboratory.** The culture is a surrogate; all biological quantities are ILLUSTRATIVE. Twin parameters (window, tolerance, dose units, KPI targets) have no laboratory source yet.
- **Symbolic durations.** On the symbolic backend durations are the planner's facts; only the Isaac runs give independent timing.
- **Concurrency.** One plate, sequential sessions; several plates under concurrency are not handled.
- **Trust gaps.** Well misregistration at L12 and lateral plate seating at L17 are only partly detected; F17 is unresolved and needs a decision on what counts as the same measurement (another field in the well, another magnification).
- **Proof scope.** Relative to A1–A6; no progress guarantee; contact physics and Giskard execution are not covered; R02's goal collisions are not checked offline.
- **Reproduction scope.** One machine and environment; no clean clone or container yet. Isaac runs are recorded inputs.
- **Competency questions.** Hand-written query functions over the trail, not yet EQL or OWL queries; no natural-language front end.

Table 42: Findings F1–F17 and the changes they caused.

#	Finding	Change
F1	The knowledge base was incomplete: plate seats came from a hard-coded name list; H4, L09, L21 had no seat facts, so a quarantine could not be grounded.	Seats from the ABox type `PlateSeatPose`, shuttles from part types; plans unchanged.
F2	Kernel and planner disagreed at s1: the kernel believed the tool still coupled 48 h later.	Freshness: observations of shared cell state go stale at a session boundary.
F3	The understanding gate caught incomplete category cards written by the assistant itself.	Cards completed; an undeclared slot blocks, a slot declared empty does not.
F4	A failing barcode read wrote no audit entry; the evidence for the escalation was lost.	Failing leaves log what they observed.
F5	The refusal for an unbound reagent was blamed on the time window.	Projection is its own constraint (`projection_succeeds`).
F6	Missing capabilities failed only after the challenge was dosed, wasting the culture.	The whole experiment is projected at acceptance.
F7	An incomplete card moved the plate before the gate refused the dose.	The gate is knowledge-only and also runs during projection.
F8	A predicted window violation was reported as ”not groundable”.	In projection, policies predict instead of abort.
F9	The lint found device names in plan code (concentration, tolerance).	Concentration is a reagent property; tolerance comes from the bound device.
F10	A failure after leaving the incubator stranded the plate at the device.	Unwind-protect at session level returns the plate first.
F11	A planning error in both derivations: QC calibration placed after the plate occupied the imager stage.	QC is a preparation step; z3 precondition ”imager free”; the old plan is refuted at step 24.
F12	Projection copied the backend’s ground truth.	Projection starts from the assistant’s belief.
F13	The twin’s devices act at contract granularity (e.g. `dose_well`).	A process-module layer binds groups of leaves to one skill; each leaf says what realised it.
F14	HermiT found 6 unsatisfiable categories: `parent` conflated specialisation, partonomy and realisation.	Cards separate `parent` (specialisation) from `parts` (refinement).
F15	The trail export needed cards for `incubate` and `contain`.	Cards added; all 222 nominal designators classified.
F16	Upstream reuse: RS `ValidityConstraint` is the validity column; AICOR’s commitment record is episode-level.	Unit `CommitmentAuditEntry`; divergence registered (D-LAT-11).
F17	Isaac `calibration_lapsed` did not end as specified: after the reload the plate sat about 93 µm off; A02 failed the focus floor twice; A05 was never imaged.	Not fixed. The focus repair (refocus) cannot change the outcome; a re-seat is prescribed and its absence is caught by the language (N9).

- **Norms.** The regulation mapping is illustrative; signature and study-plan approval are documentary only (Part 11 not covered).
- **Pending decisions.** K1 (rule vs norm set), K4 (warrant definitions), K10 (epistemic vocabularies); calibration freshness; whether a human sign-off model is needed.

17 Roadmap

- **coraplex port.** `ActionDesignator` ⟶ `ActionDescription`, leaves ⟶ `MotionNode`; a device-command executable is proposed; resolution queries ⟶ krrood EQL over the semantic digital twin (lab annotations for plate, well, nest, device); CQs ⟶ `cram-viz-check` specifications. LDL contracts become pre- and post-conditions; `prevents` and `maintains` need a monitor attachment (D-LAT-11).
- **Re-seat repair and Isaac re-run.** Implement the prescribed re-seat for misseated plates at L17, confirm the plate movement from the physics state, re-run `isaac_calibration_lapsed`.
- **Close the trust gaps.** A dispense-position observation at L12; a lateral seating observation at L17.
- **Clean-room reproduction.** Container image and a clean clone; re-run `--verify` there.
- **Science.** A priming design to separate M2 from M3; several plates under concurrency.
- **Giskard execution.** A Cell 01 SDT world so the motion module runs; attach and detach the plate between statecharts; compliant insertion.

References

[1] Drew McDermott. A reactive plan language. Technical Report YALEU/CSD/RR 864, Yale University, Department of Computer Science, 1991.

[2] Michael Beetz. Structured reactive controllers. *Autonomous Agents and Multi-Agent Systems*, 4(1–2):25–55, 2001.

[3] Michael Beetz, Lorenz Mösenlechner, and Moritz Tenorth. CRAM – a cognitive robot abstract machine for everyday manipulation in human environments. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1012–1017, 2010.

[4] Simon Stelter, Georg Bartels, and Michael Beetz. An open-source motion planning framework for mobile manipulators using constraint-based task space control with linear MPC. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.

[5] Ross D. King, Jem Rowland, Stephen G. Oliver, Michael Young, et al. The automation of science. *Science*, 324 (5923):85–89, 2009.

[6] Kevin Williams, Elizabeth Bilsland, Andrew Sparkes, et al. Cheaper faster drug development validated by the repositioning of drugs against neglected tropical diseases. *Journal of the Royal Society Interface*, 12(104): 20141289, 2015.

[7] Benjamin Burger, Phillip M. Maffettone, Vladimir V. Gusev, et al. A mobile robotic chemist. *Nature*, 583:237–241, 2020.

[8] Milad Abolhasani and Eugenia Kumacheva. The rise of self-driving labs in chemical and materials sciences. *Nature Synthesis*, 2:483–492, 2023.

[9] Monya Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016.

[10] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, et al. The fair guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016.

[11] U.S. Food and Drug Administration. Data integrity and compliance with drug CGMP: Questions and answers. guidance for industry, 2018.

[12] Dennis V. Lindley. On a measure of the information provided by an experiment. *The Annals of Mathematical Statistics*, 27(4):986–1005, 1956.

[13] Kathryn Chaloner and Isabella Verdinelli. Bayesian experimental design: A review. *Statistical Science*, 10(3): 273–304, 1995.

[14] Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. Multi-shot ASP solving with clingo. *Theory and Practice of Logic Programming*, 19(1):27–82, 2019.

[15] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 4963 of *LNCS*, pages 337–340, 2008.

[16] Daniel S. Weld. An introduction to least commitment planning. *AI Magazine*, 15(4):27–61, 1994.

[17] Stuart Haber and W. Scott Stornetta. How to time-stamp a digital document. *Journal of Cryptology*, 3(2):99–111, 1991.

[18] Martín Abadi and Leslie Lamport. The existence of refinement mappings. *Theoretical Computer Science*, 82(2): 253–284, 1991.

[19] Albert Benveniste, Benoît Caillaud, Dejan Nickovic, et al. Contracts for system design. *Foundations and Trends in Electronic Design Automation*, 12(2–3):124–400, 2018.

A Experimentation plan tables

Table 43: Failure handlers of the experiment's control program (LDL).

Failure	Handler (LDL)
identity-mismatch	(on (a failure (type identity-mismatch)) (do (an action (type quarantining) (material (the plate (identity mismatched))))) (then escalate) (guarantees safe-state contained))
measurement-invalid	(on (a failure (type measurement-invalid) (cause calibration_expired)) (do (re-establish (an instrument (function imaging) (calibration current)))) (then replan) (else escalate) (guarantees goal-or-safe-state))

Failure	Handler (LDL)
measurement-invalid	(on (a failure (type measurement-invalid) (cause out_of_focus)) (do (re-establish (the plate ?p (seated true)))) (then (retry (max 1))) (else escalate) (guarantees goal-or-safe-state))
measurement-invalid	(on (a failure (type measurement-invalid) (cause plate_misseated)) (do (re-establish (the plate ?p (seated true)))) (then (retry (max 1))) (else escalate) (guarantees goal-or-safe-state))
validity-effect-unachievable	(on (a failure (type validity-effect-unachievable)) (then escalate) (guarantees safe-state))
dose-not-delivered	(on (a failure (type dose-not-delivered) (cause wrong_reagent)) (do (re-establish (some substance ?s (identity verified)))) (then (retry (max 1))) (else escalate) (guarantees goal-or-safe-state))
dose-not-delivered	(on (a failure (type dose-not-delivered) (cause tip_missing_or_dropped)) (then (retry (max 1))) (else escalate) (guarantees goal-or-safe-state))
dose-not-delivered	(on (a failure (type dose-not-delivered) (cause clog_or_air_aspiration)) (then escalate) (guarantees safe-state))
dose-not-delivered	(on (a failure (type dose-not-delivered) (cause well_misregistration)) (do (an action (type quarantining) (material ?p))) (then escalate) (guarantees safe-state contained))
contamination-suspected	(on (a failure (type contamination-suspected)) (do (an action (type quarantining) (material ?p))) (then escalate) (guarantees safe-state contained))
environment-excursion	(on (a failure (type environment-excursion)) (do (restore (the plate ?p (in ?env)))) (then escalate) (guarantees safe-state))
containment-breach	(on (a failure (type containment-breach)) (then escalate) (guarantees safe-state))
prevents-violated	(on (a failure (type prevents-violated)) (do (restore (the plate ?p (in ?env)))) (then escalate) (guarantees safe-state))

B Competency questions

Table 44: The competency questions and the scenarios for which an expected answer was checked.

CQ	Family	Question	Scenarios checked
CQ-01	what	What are you doing at the first dispense – which device, plate, well, reagent and volume?	nominal, tip lost
CQ-02	why	Why are you dispensing into this well?	nominal
CQ-03	why	Why was this device chosen for dosing, and which alternatives were rejected?	nominal
CQ-04	why	Why does the plate go via the handover nest?	nominal
CQ-05	why	Why does R02 couple its tool in every session?	nominal
CQ-06	how	How long was the plate outside the incubator in each session, and what is the limit?	nominal, uncalibrated known
CQ-07	constraints	Which constraints were checked in session s0, with which verdicts?	nominal
CQ-07b	constraints	Which constraints were violated in session s1?	tool changer slow
CQ-08	constraints	What must hold before and while you dispense?	nominal
CQ-09	capability	Which devices can image the plate now, and how far is their readout validated?	nominal, imager swapped, imager incapable
CQ-10	capability	Can R02 carry the plate from L10 to the handover nest?	nominal
CQ-11	what-if	What would change in the imaging session if L17 were not known to be calibrated?	nominal
CQ-12	what-if	What would happen if a tool change took 600 s?	nominal
CQ-13	what-if	What would happen if contamination were suspected?	nominal
CQ-14	evidence	How do you know that A01 received the challenge?	nominal
CQ-15	evidence	Is your record intact, and would you notice if someone edited it?	nominal, identity swap, imager swapped
CQ-16	retrospective	What went wrong, and what did you repair?	nominal, calibration lapsed, tip lost, identity swap, tool changer slow
CQ-17	retrospective	Was the rescue given? If not, why not?	nominal, tip lost, identity swap, tool changer slow, rescue unbound
CQ-18	retrospective	Where is the experiment's plate now, and how do you know?	nominal, identity swap, control dropped
CQ-19	normative	Which regulation clauses does this run cover, which not?	nominal, contamination
CQ-20	normative	Which material did you quarantine, and who authorised it?	nominal, identity swap
CQ-21	verdict	Is the result a valid experiment?	nominal, calibration lapsed, imager swapped, identity swap
CQ-22	verdict	May this result update the culture belief?	nominal, calibration lapsed, imager swapped, identity swap
CQ-23	epistemic	What do you not know?	nominal, identity swap
CQ-24	what	What did R02 do in the rescue session?	nominal
CQ-25	taxonomy	What kind of action is a dispense, and what does it realise?	nominal
CQ-26	modularity	If L17 were exchanged for the generic imager, what would change?	nominal

C Glossary

Table 45: Terms used in this report.

Term	Meaning in this report
action designator	a symbolic description of an action, resolved against knowledge at run time (CRAM)
entity designator	a partial description of a lab entity: (determiner sort slot*), answered by a knowledge query
culture plan	the experiment over the cultures only: actions with physical and knowledge effects
lab plan	LDL action designators with contracts, instruments, handlers and KPIs
step plan	the grounded robot and device steps, derived by clingo and expanded by the kernel
session / trip	one excursion of the plate from the incubator, a realization concept
realization / validity	the two columns of the verdict: ran correctly and safely / result interpretable
NR3	realised but not valid; nothing may be committed
EIG	expected information gain of a design over the competing models (bits)
grounds	the fact atoms a resolution or answer used
ILLUSTRATIVE	a twin value or mapping, labelled; not a lab or biomedical fact (D016)
Cell 01	the two-arm robotic cell of the laboratory twin (Isaac Sim, OpenUSD)

D Rebuilding this report

All commands run from the repository root with the project's Python environment:

- `env_cad/bin/python -m rsa.reproduce --verify` re-runs the 12-stage derivation and compares manifests;
- `env_cad/bin/python paper/build_paper.py` builds the paper;
- `env_cad/bin/python report/build_report.py` builds this report: `report/rsa_report.tex` and, with LuaLaTeX, `report/rsa_report.pdf`.

This build read the outputs at manifest `a8a39851a44328aa` (full hash on the title page). The report's TeX source needs `refs.bib` and the `figures/` directory beside it; fonts are Libertinus Serif, IBM Plex Sans and IBM Plex Mono.